

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

October 8, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	October 8, 2026	53 pages, 6 appendix pages

Tiivistelmä — Referat — Abstract

Radio access network (RAN) control planes are written mostly in C++ for performance, at a well-known cost in memory safety and concurrency. Rust promises comparable performance with compile-time safety, but its fit for this latency-critical setting is unproven.

The thesis compares modern C++ and Rust using a user equipment (UE) Context Management service that keeps per-device state, as a 5G base-station control plane (gNB-CU-CP) does. Average-throughput comparisons leave four dimensions open: tail latency, the defect classes Rust prevents at compile time, safety-model overhead, and incremental adoption.

One language-agnostic specification is frozen and then implemented in both C++ and Rust. The two versions use matched algorithms, the same concurrency model, allocator, and build settings. Forced-idiom gaps are logged. The synthetic workload is increased until overload. Both versions pass the same parity tests. The evaluation also adds fault injection and a foreign function interface (FFI) prototype.

Safe Rust refuses to compile code that contains any of the seven injected memory-safety and data-race defects, whereas C++ compiles the same code. Both languages compile code that contains seven further defects, such as deadlocks.

Each version was run with two memory allocators, the system default and `jemalloc`. Rust answers a typical request up to 1.5 μs faster than C++. The difference appears while the service keeps up. The service's own work is only 0.250% of the time taken by a request. At 784 thousand requests per second, Rust completes more requests with both allocators. When several requests are in progress at once, the wait depends on the allocator. The wait is shorter for Rust with `jemalloc` and for C++ with the system allocator.

On all three machines tested, Rust uses a few MiB more peak memory than C++ with both allocators. Existing C++ code can call the unchanged Rust service through a thin C interface. The interface adds at most 10.91% to one request.

ACM Computing Classification System (CCS)
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, incremental adoption

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

List of Figures **iii**

List of Tables **iv**

List of acronyms **v**

Key terms **vi**

1 Introduction **1**

1.1 Motivation 2

1.2 Industrial context 3

1.3 Problem statement 4

1.4 Research questions and hypotheses 5

2 Background: Systems Languages for Latency-Critical Software **7**

2.1 RAN control-plane systems and the gNB-CU-CP 7

2.2 Research gap 8

3 Research Methodology **11**

3.1 The UE Context Management subsystem 11

3.2 The defect catalogue 12

3.3 Research approach 13

3.4 Methods per research question 15

3.5 Fairness as a controlled variable 16

3.6 Statistical treatment 17

3.7 Reproducibility and research artefacts 18

4 Design and Implementation **19**

5 Experimental Setup and Evaluation **21**

5.1 Testbed setup 21

5.2 Workload and measurements 23

5.3 Results . . . 24
5.3.1 Below saturation (RQ1, RQ3) . . . 24
5.3.2 Past saturation (RQ1) . . . 28
5.3.3 Resource use (RQ3) . . . 30
5.3.4 Safety via fault injection (RQ2) . . . 32
5.3.5 Migration (RQ4) . . . 34

6 Discussion **36**
6.1 Summary of findings . . . 36
6.2 Discussion on the research questions . . . 37
6.3 Revisiting the hypotheses . . . 40
6.4 Engineering guidelines for incremental adoption . . . 41
6.5 Where this study sits . . . 41
6.6 Ethical and professional reflection . . . 42

7 Conclusion and Future Work **43**

Bibliography **45**

A Host and campaign configuration

List of Figures

2.1 Where the studied subsystem sits in the 5G NG-RAN 8
2.2 A checklist of earlier comparisons 10

3.1 UE-context lifecycle state machine 11

4.1 Shared-nothing, thread-per-shard architecture 19

5.1 The two test workloads 23
5.2 The load sweep: tail latency and achieved throughput 26
5.3 Where the end-to-end clock goes 27
5.4 How small the language's share of a round-trip is 27
5.5 Four load points, from a light load to overload 28
5.6 Peak memory and CPU of the four builds 32
5.7 Where the guarantee stops: the outcome matrix 34
5.8 The thin layer where C++ calls the Rust service 35

6.1 Where Rust's compile-time guarantee stops 39

List of Tables

3.1 Injected defects and their predicted outcomes 13
3.2 How each research question is answered 15

5.1 How long one call takes at the chosen load 25
5.2 Delay with several requests in flight 30
5.3 Extra peak RAM on each host 30

A.1 The three measurement hosts ii
A.2 Configuration of the service, the workload, and the campaign iii
A.3 How much the delay changes from one call to the next v
A.4 Where the process's extra memory sits vi

List of acronyms

Each acronym below is also written out where the text first uses it. Terms that are not acronyms, such as *tail latency* and *shard*, are collected in the list of key terms that follows.

Acronym	Expansion
3GPP	3rd Generation Partnership Project, the body that specifies the 5G standards
ABI	Application binary interface
AI	Artificial intelligence
AI-RAN	An architecture in which AI and radio access network workloads share one accelerated infrastructure
API	Application programming interface
ASN.1	Abstract Syntax Notation One
CPU	Central processing unit
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
DPDK	Data Plane Development Kit
EU	European Union
FFI	Foreign function interface
FIFO	First in, first out
FNV-1a	Fowler–Noll–Vo hash function, variant 1a
GCC	GNU Compiler Collection
gNB	Next-generation NodeB, the 5G base station
gNB-CU	gNB central unit
gNB-CU-CP	gNB central unit control plane, the domain this work sits in
gNB-CU-UP	gNB central unit user plane
gNB-DU	gNB distributed unit
HDR	High dynamic range, of the latency histogram
IMT-2030	International Mobile Telecommunications-2030, the ITU framework for 6G
ITU	International Telecommunication Union
KPI	Key performance indicator
kops/s	Kilo-operations per second
KVM	Kernel-based Virtual Machine
MPSC	Multi-producer, single-consumer
NG-RAN	Next-generation radio access network
PHY	Physical layer
RAII	Resource Acquisition Is Initialization
RAN	Radio access network
RQ	Research question
RRC	Radio Resource Control
RSS	Resident set size
UE	User equipment, an end device such as a phone or a modem
UECM	UE Context Management, the subsystem reimplemented and measured here

Key terms

The table below gives plain meanings for the network, the service, the timing, and the safety words used in the thesis.

Term	Meaning
Network	
Control plane	The messages that set up and control a call. The user's data travels on the user plane
User plane	The path that carries the user's data. The control plane sets that path up and takes it down
Signalling	The messages that create, change, and end a connection
Handover	Moving one connected device from one cell to another, so the call can continue
Cell	One coverage area of the radio network
Mobility	A connected device changing cell while the call continues
UE context	The stored record for one device. A handover or a release changes that record
Reconfiguration	A change to a connection that stays up, such as adding a data channel
Release	Ending one device's connection and deleting its record
Procedure	One action a person would notice, such as setting up a call or moving between cells. It is made of many smaller steps
Open-RAN	The same split base station, opened so each part may come from a different company and run as ordinary software
5G core	The network behind the base station. It forwards each call onward
The service	
Shard	One slice of the device records. Exactly one thread owns that slice
Producer	A thread that sends the test's requests and waits for each reply
Worker	The thread that owns one shard and carries out the requests for it
Bounded queue	The queue in front of a shard. It has a fixed size and waits when full, so a service that cannot keep up shows delay rather than dropped requests
Operation	One request for one device, such as setup or release
Variant	One of the four builds under test. The four are C++ or Rust, each linked with one of the two memory libraries

Term	Meaning
Allocator	The library that hands out memory and takes it back
Timing	
Tail latency	How long the slow calls take, rather than the average call. This thesis reads that from the p99, the p99.9, and the p99.99
Median	The middle call, once the calls are ordered by how long they took. Half are faster and half are slower
Histogram	The record of how long each call took. Tail latency is read from its slow end
Offered load	The rate the test tries to send, counted whether or not the service answers in time
Throughput	How many requests the service finishes each second. Offered load is how many the test tries to send
Saturation	The load at which the service can no longer keep up and requests start to wait
Overload	Load past saturation. Requests wait because the service cannot keep up
Round-trip	The time from the planned send until the reply comes back. It includes the wait in the queue
Bare metal	The program runs on the computer itself, with no virtual machine between it and the hardware
Guest	A virtual machine an earlier measurement ran in. The reported results come from the bare-metal computer
Safety	
Data race	Two threads use the same memory at once and at least one writes, with nothing to keep the two uses in order
Race condition	A bug that depends on which of two steps finishes first. Safe Rust still compiles these
Use-after-free	Using memory after it has been given back
Double-free	Giving the same memory back twice
Use-after-move	Reading from a value after its contents have been moved elsewhere. The old name is still valid but empty
Ownership	Rust's rule that each value has one owner. A read of a record whose owner is already gone does not compile
Borrowing	A temporary loan of a value. The compiler rejects a loan that lasts longer than the value
Unsafe	A region the programmer marks so the Rust compiler stops checking it. C++ has no such mark
Sanitiser	A tool that watches a running C++ program and reports some defects. It can report only a path a test actually runs

1 Introduction

Mobile networks are moving from specialised hardware to software on ordinary servers. Functions that once ran on fixed-purpose, vertically integrated equipment are now reimplemented as programs on general-purpose machines. The radio access network (RAN), long the most hardware-centric part of the stack, is part of that shift.

In 5G, the base station, formally the next-generation NodeB (gNB), is split into a central unit and one or more distributed units. The central unit is split again into control-plane and user-plane functions [1, 2]. The control plane carries the signalling, the messages that set up and control the call. The user plane carries the phone's data. Open-RAN pushes the same idea further: the RAN becomes virtualised, multi-vendor software linked by open interfaces [3]. The control plane that results, the gNB central unit control plane (gNB-CU-CP), is a large, stateful, highly concurrent program. It tracks every connected device. The standards call each device a user equipment (UE). The control plane runs Radio Resource Control (RRC) signalling [4] and coordinates mobility and handovers, all within strict latency limits.

A single site can run thousands of connection-management and mobility procedures per second. Each procedure has to complete within the millisecond-scale budget the 5G system sets for control-plane signalling [5]. A deployed control plane may track millions of active devices at once. As the RAN moves to cloud-native, disaggregated deployments, the correctness, reliability, and timing of this software increasingly set the quality of the whole network. An outage here is not a local defect. Neither is a tail-latency spike: a few calls take much longer than the rest, even when the average still looks fine. Every device the site serves is affected [6, 7].

This control-plane software is, today, written mostly in C++. The choice is a practical one: C++ offers the low-level control, mature tooling, and performance that latency-critical systems demand. But that performance comes with a well-known cost. C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [8, 9]. Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [10, 11], with performance intended to be compet-

itive with C++ [12]. Whether that promise holds for a telecom-grade, latency-critical control plane, where predictability under overload matters as much as raw speed, is not well understood. This thesis, carried out at Nokia, addresses the question directly: it develops two functionally equivalent implementations of a representative RAN subsystem, a UE Context Management service that keeps per-device state, as a gNB-CU-CP does, in modern C++ and in Rust, and compares them under a realistic synthetic signalling workload.

Three terms recur throughout, and the first is the *gNB-CU-CP*, the control-plane class this work sits in. *UE Context Management (UECM)* is the compact service reimplemented and measured. *Tail latency*, the p99, p99.9 and p99.99 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its latency budgets [6, 7].

1.1 Motivation

Three trends make this comparison timely. The first is the cloud-native evolution of the RAN. Control-plane functions now run on general-purpose servers, so the software is longer-lived, more concurrent, and must meet its latency budgets without the determinism of purpose-built hardware. Open-RAN disaggregation turns the base station into virtualised, multi-vendor software [3]. A latency regression or a memory-corruption defect is then a network-wide risk, and the language choice is an engineering decision.

The second trend is artificial intelligence (AI). AI used to be only a tool. It helps to choose radio settings, such as how much power to send. That tool does not have to run on the radio's own server. Now AI programs also run on that server, next to the radio software [13]. Both run at the same time, so they share one processor. International Mobile Telecommunications-2030 (IMT-2030) is the 6G framework of the International Telecommunication Union (ITU). It lists AI together with communication as one named job for 6G [14]. The architecture forming around that job is AI-RAN, in which AI and radio work share one accelerated server [15]. A control plane there no longer has the host to itself. It shares the central processing unit (CPU), memory, and cache with inference work it does not schedule. When that neighbour takes capacity, the control plane saturates sooner: it stops keeping up with demand at a lower load than it would alone. Chapter 5 finds the widest campaign-clock gaps past that point. Later diagnostic sweeps do not reproduce either gap, so they cannot be read as a ranking of the two services.

The third is the shift toward memory-safe languages. Large software organisations report

that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages reduces them [8, 9, 16, 17]. Independent work argues that memory safety now needs standardised requirements if it is to be procured at all [18]. Cybersecurity agencies urge a move away from C and C++, and the European Union (EU) Cyber Resilience Act adds legal duties from September 2026 and December 2027 [19, 20, 21]. The Act names no programming language. Direct evidence for a RAN control plane is still missing. Providing it is the motivation for this work.

1.2 Industrial context

This thesis is carried out at Nokia. The company is one of the small number of vendors that build, maintain, and evolve production RAN software end to end. The five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high [22, 23]. The subsystem it studies, UECM, is a compact model of the per-UE state work a gNB-CU-CP does.

In the architecture of the 3rd Generation Partnership Project (3GPP), the gNB-CU-CP is the control-plane half of the central unit. It hosts RRC and manages the UE context lifecycle [1, 4], under the millisecond-scale budgets the 5G system sets for control signalling [5]. Chapter 2 places that control plane in the split 5G base station, the gNB. Because it is written in C++, it inherits that language's performance and its memory-safety and concurrency hazards. The three trends of Section 1.1 meet here. Software of this class runs on live operator networks. The language choice is a production concern.

UECM is small enough to reimplement faithfully and reason about in full. It still carries the properties that make control-plane code hard. The implementation follows a public, frozen specification, not proprietary code, so readers outside Nokia can reproduce the comparison.

A decision with a generational horizon. A language choice in software of this class outlives the generation it was made for. 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work. The ITU asks for technology proposals in early 2029 and for specification submissions by mid-2030 [24]. That work covers migration from 5G and interworking with it. A control plane written now is maintained alongside the next generation, not retired. IMT-2030 still names security and resilience as a design principle and still carries the ultra-reliable low-latency scenario in strengthened form [14].

The measurements here use a current-generation subsystem and published requirements.

No claim is made about requirements that have not been written.

A concrete failure mode. A moving train full of passengers stands for the connected UEs at one site. A handover moves one passenger from one cell to another, and the call can continue. Each passenger has a stored record, the UE context. When many passengers move at once, the control plane is under a mobility storm. One passenger leaves, so that record is deleted, while a message for that passenger is still being handled. Which of those two steps finishes first is a race condition. If the read comes after the delete, it is a use-after-free. In C++ this still compiles. Tests that do not hit that ordering can pass. When many calls run at once, the bug can corrupt another passenger's record. That race can crash the thread that is serving the call. A few calls can become much slower than the rest. That jump in the slowest calls is a tail-latency spike. In safe Rust the same design does not compile: ownership and borrowing reject the read of a record that has already been deleted. That is why this thesis measures both *safety* (which defects are prevented, and when) and *predictability* (tail latency under overload), not average throughput alone.

1.3 Problem statement

Two properties of this domain make the language choice consequential. First, manual memory and concurrency management in C++ exposes the software to defects such as use-after-free, double-free, data races, and iterator invalidation. These defects are silent at compile time and costly to find and fix at telecom scale. Second, what governs whether RAN latency budgets are met is not average throughput but worst-case and tail behaviour under load and overload [6], a property that is easy to regress and hard to measure correctly. Rust promises to remove the first class of defect at compile time while claiming performance competitive on the second, but that claim rests mostly on general-purpose benchmarks rather than on stateful, concurrent, latency-critical telecom workloads.

Bringing these threads together, the concrete problem this thesis addresses is the following. Telecom vendors are under growing pressure, both technical and increasingly regulatory, to adopt memory-safe languages, yet for the RAN control plane they lack *direct, controlled* evidence on which to base that decision. It is not established whether a Rust implementation of a stateful, concurrent control-plane subsystem can match modern C++ in *tail latency and predictability under overload* (as opposed to average throughput). It is also unknown whether Rust's compile-time guarantees actually eliminate the specific defect classes that affect the existing C++ code, and what incremental adoption *costs* inside a large C++ codebase. The available evidence is largely general-purpose and average-

case [12, 25], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem. Answering them under controlled, reproducible conditions is the problem this thesis sets out to solve.

1.4 Research questions and hypotheses

The thesis is organised around four research questions. They put latency predictability first, then defect scope, then resource cost, then incremental adoption:

RQ1 How do Rust and modern C++ compare in latency predictability (median, p99, p99.9 and p99.99 tail latency, and the maximum) for a RAN control-plane workload under increasing load and overload?

RQ2 Which classes of memory-safety and concurrency defects that occur in C++ are eliminated at compile time in the Rust implementation, and at what stage and cost are the remaining ones detected?

RQ3 What is the overhead, if any, of Rust's safety model (bounds checks, ownership-driven copies, abstraction cost) on throughput, CPU utilisation, and memory behaviour?

RQ4 What are the practical implications (effort, foreign function interface (FFI) interoperability, and risk) of adopting Rust incrementally within an existing C++ RAN software ecosystem?

Three hypotheses are written down before any measurement.

H1 For this workload the two languages are performance-equivalent below saturation, with any difference only in the far tail.

H2 Safe Rust prevents at compile time the memory-safety and concurrency defect classes that C++ only tries to catch after the program is built.

H3 Calling the Rust service from C++ through the FFI adds only a small share of one signalling call's time. The real cost of adding Rust a piece at a time is engineering work, not slower calls.

Chapter 6 revisits all three hypotheses against the facts.

The RQ2 list has fourteen defect classes. Safe Rust is expected to stop seven of them at compile time. It is expected not to stop the other seven. That split is the boundary the study looks for, not only a check that the guarantee works. The seven boundary classes are lock-order deadlock, lost update, reference-cycle leak, release-build overflow, worker-failure blast radius, seam double-free, and forgotten lock guard.

A previous study already estimated that boundary from a list, not from programs that were built. It took the Common Weakness Enumeration (CWE), a long list of bug types. It marked each type as one that safe Rust would stop or would not. It then counted real product bugs in 68 open-source C projects. 58.2% of those classified bugs sat in types that safe Rust would have made almost impossible [26]. That share is a best case: it assumes the new code stays in safe Rust. A type counts as stopped if the mapping says so, even if no compiler has rejected a program. RQ2 asks the same question of programs that are built and run. That is how it can say when the types still open are caught, and what it costs to catch them.

2 Background: Systems Languages for Latency-Critical Software

2.1 RAN control-plane systems and the gNB-CU-CP

The next-generation radio access network (NG-RAN) is the setting for this comparison. Event rates are high: UEs move, and signalling can spike. Latency bounds are tight and must stay stable. The 5G service and RAN specs state those bounds as key performance indicators (KPIs) [5, 27]. The software lives for a long time and keeps growing. Many threads run at once. Those bounds are on the latency of a signalling call. A late call still misses them when the average call is on time. So the worst case and the tail of that latency matter more than the mean. That is why this thesis measures the tail.

The gNB is split on two axes. The first split is central versus distributed. The gNB runs a protocol stack: a set of layered radio protocols from the air interface up to RRC signalling. A central unit (gNB-CU) hosts the higher layers. One or more distributed units (gNB-DU) host the lower layers. Those layers sit close to the radio and have tight low latency requirements. The second split is control versus user plane, inside the central unit. It splits the control-plane function (gNB-CU-CP) from the user-plane function (gNB-CU-UP). These parts talk over standard links. Uu is the radio link from the device. NG goes toward the 5G core, F1 sits between the CU and the DU, and E1 sits between the CU-CP and the CU-UP [1, 2, 28]. Open-RAN pushes the same links further. Each part can then come from a different vendor and run as software on ordinary machines [3]. Figure 2.1 shows the part studied in this thesis, UE Context Management (UECM), inside the gNB-CU-CP. UECM owns the per-UE control-plane state. It also handles RRC signalling and mobility. These are integral control-plane functions, which makes UECM a representative stateful subsystem. It does not stand for the control plane as a whole.

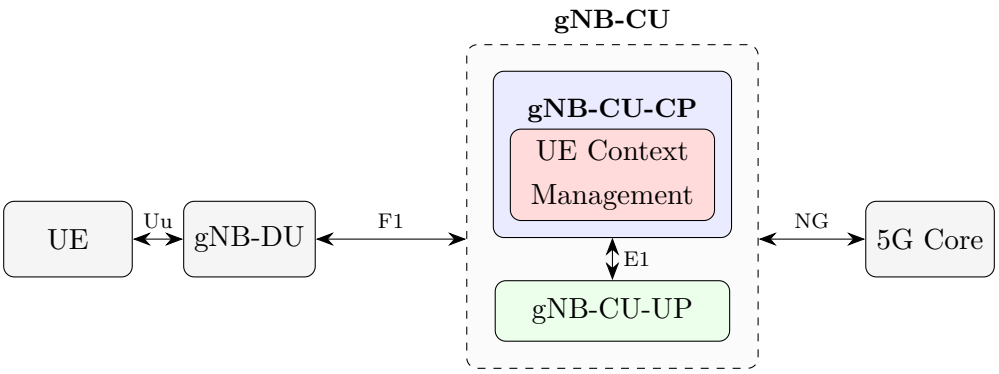


Figure 2.1: Where the studied subsystem sits in the 5G NG-RAN. UE Context Management is highlighted inside the gNB-CU-CP.

2.2 Research gap

The RAN control plane has many threads and tight latency bounds. Tail latency under too much load, not mean throughput, sets the quality the network delivers. C++ gives the control and speed the domain needs. But it leaves whole classes of memory and thread bugs to detection at runtime. Rust promises to block those classes of errors at compile time at no run-time overhead. It also offers a step-by-step adoption path through the C application binary interface (ABI).

Five vendor-hosted Master's theses, all done at Ericsson, have compared candidate languages for RAN and telecom software. One weighs run-time cost against how fast people can build, across C++, Rust, and Go [29]. One asks whether Rust could replace C in RAN node management [30]. One builds a packet processor, one thread per core, in async Rust. It measures that processor against the Data Plane Development Kit (DPDK) program it mirrors [31]. One measures median and p99 latency for control-plane steps across five async frameworks [32]. The last builds matched C++ and Rust versions of one program. That program turns a packed Abstract Syntax Notation One (ASN.1) message back into a readable form. It compares them with statistical tests [33]. All five are directly relevant. They do not all favour Rust. Olsson and Stenström's packet processor reaches a fifth to a half of DPDK's throughput, with far worse latency [31]. Arvidsson and Kaller's Rust p99 rises more steeply past a threshold rate [32]. Karlbäck and Orö pick C++ where speed is critical. They still find Rust lightest on peak memory [29]. Olsson's management software is as fast as C or faster. It uses more peak memory [30]. Emberg and Wadsten's pair is comparable in one session. It is 85–91% worse in the next [33].

Three of them change the language together with the run-time and the thread model. A

measured gap then cannot be blamed on the language rather than on the framework [29, 31, 32]. In that packet-processor study the DPDK program does two to five times as much work [31]. A compiler does not move throughput that far. The run-time does. Olsson's management study and Emberg and Wadsten do not change the run-time with the language [30, 33]. But they report run time and peak memory, not tail latency under rising load. Emberg and Wadsten come closest in method to this thesis. Their program keeps no state between calls. The service in this thesis stores state and uses many threads. Figure 2.2 places the five vendor theses beside the general-purpose comparisons [34] and the matched systems-software pairs [35, 36]. Each of these groups leaves out at least one thing the figure asks for. No group keeps the design fixed, measures the tail, and drives the service into overload.

The place these studies come from is part of the gap too. All five studies come from one vendor. Their conclusions have never been tested against another firm's software, targets or house rules. So what is known about this question is what one firm's frameworks say about it. The published record on the other side is thinner. There is no study of Rust among the indexed papers of the vendor hosting this work. The nearest peer-reviewed stance from that side argues for making memory-error sanitisation practical. It does not argue for replacing it with a compile-time guarantee [37]. That is the strategy this thesis tests against.

What is missing from prior work is a direct, controlled test of a typical control-plane part. The open questions are these. Does Rust hold its tail latency under too much load against modern C++ when the thread model, operations, and data layout stay fixed? Which bug classes does it remove, and at what stage are the rest caught? What does its safety model cost in throughput and in CPU and memory? What does step-by-step adoption cost in practice? Closing this gap is the purpose of the four research questions. The next chapter grounds them in the part under study, the bug classes, and the timing at stake. It also sets out the method.

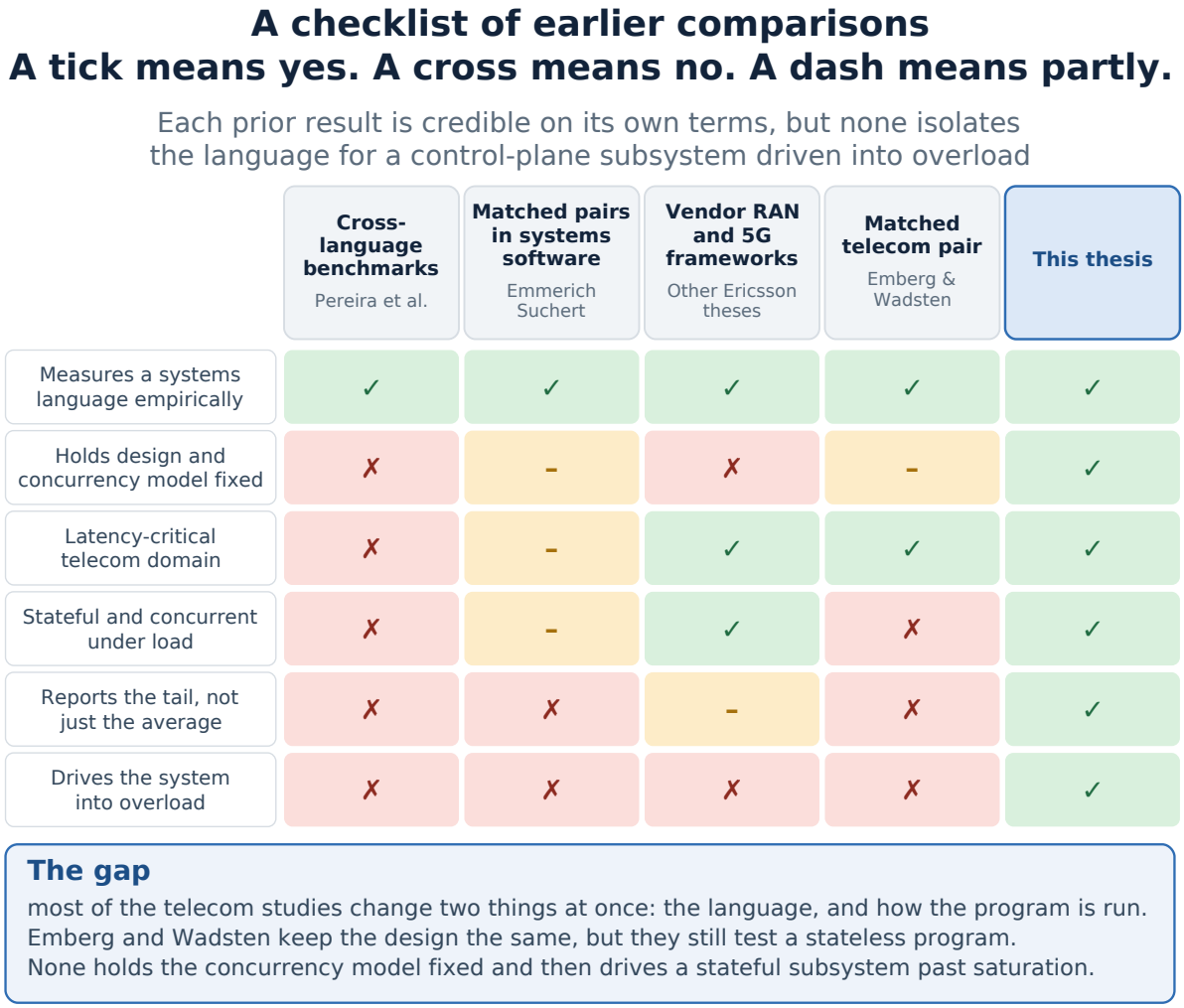


Figure 2.2: A checklist of earlier comparisons. Each row is a property a controlled answer would need, and each column is a group of studies. The vendor-RAN column is the other Ericsson theses. Emberg and Wadsten have their own column because they keep the design the same in both languages.

3 Research Methodology

The previous chapter said what earlier studies leave out. This chapter says how the study closes that gap.

3.1 The UE Context Management subsystem

The part under study is a UE Context Management service. Its operations follow the UE-context lifecycle, RRC procedure execution [4], and mobility and handover that the 3GPP design assigns to the control-plane part of a split gNB (Section 2.1). Figure 3.1 shows the UE-context lifecycle both versions encode the same way. The mixed workload of Section 5.2 walks each UE through that machine.

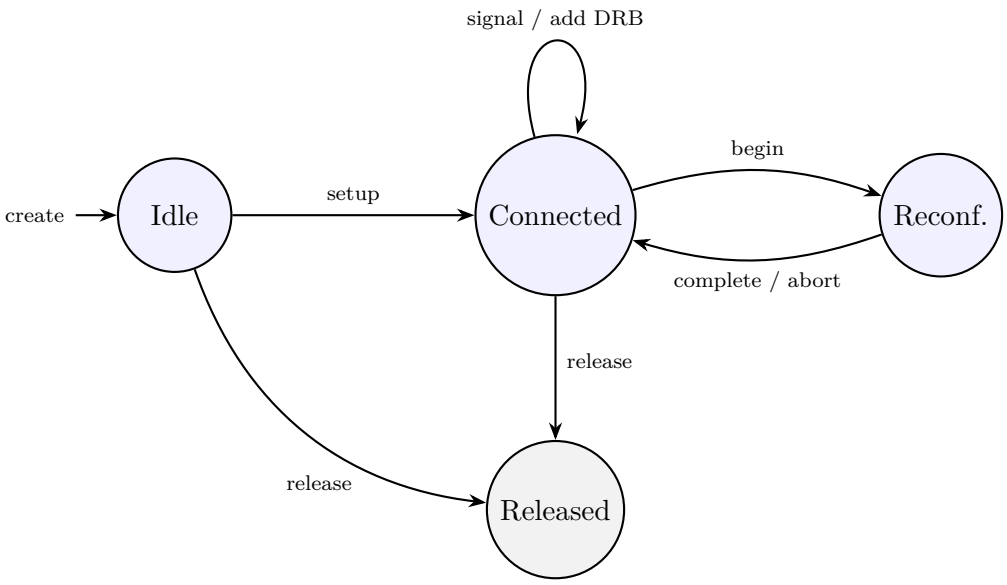


Figure 3.1: UE-context lifecycle state machine. Operations invalid for the current state return an error instead of changing state.

UECM is core RAN layer-3 work. It has the traits that make live control-plane code hard. Each UE has state that changes, the operation lifecycle is strict, and many UEs run at once. A delay target is the longest a control-plane procedure may take. The 5G service requirements name latency as one such limit [27]. For the move from a battery-saving idle state to the start of sending data, that limit is 10 ms [5]. The procedure is a chain of steps. It is late if the chain is late, even when most steps are fast. Findings about how a language handles those traits are most relevant to other stateful, multi-thread control-

plane services of this kind. The part is also small enough to specify in full and build twice. That is what makes a parity-tested comparison possible.

The interface is small and built around the lifecycle. It can create and release a context, set up a connection, handle an in-session signalling message, add a data radio bearer, begin, complete, or abort a reconfiguration, and look up a context's state under load. Each operation is defined for each state of Figure 3.1. An illegal operation returns a typed error and leaves the context unchanged. That closed set lets the same tests and the same workload drive both versions along the same paths.

RQ2 asks which defects safe Rust stops at compile time, and which ones it does not. Section 3.2 sets out the catalogue. For each defect, the expected result was written down before the programs were run. Section 5.3.4 reports what each one did.

The study measures the tail. The tail is the slow calls, not the ordinary ones. Two facts about this part make those slow calls the right thing to measure.

The first fact is what a phone user waits for. Setting up a call, or moving a phone to the next cell, is not one step. It is a chain of control-plane steps, and they have to finish in order. The user is still waiting until the slowest step in that chain is done. A busy site runs thousands of these chains every second. A call that is slow only rarely therefore still happens often enough to notice.

The second fact is about the test clock. The sender follows a fixed timetable. A late reply does not hold back the next request. That rule is called an open-loop clock. Offered load is how fast the timetable sends work. Achieved rate is how fast the service finishes it. While the timetable is gentle, the two rates match and the delay stays small.

When the timetable is made faster, a limit appears and the delay jumps. That limit can be the service, or work the sender still does while the timer is running, such as writing down the late replies. Because either one can cause the jump, the study sends more work than the service can finish and then checks which of the two the timer was counting.

3.2 The defect catalogue

The defects considered in this study fall into two broad families: *memory-safety defects* and *concurrency defects*. This classification is useful because the two families capture different ways in which C++ control-plane code can fail, and because safe Rust is expected to provide different guarantees for them.

Table 3.1 lists the 14 classes injected in Chapter 5, each in the form it would take in UE

Context Management, with the outcome expected of safe Rust before the study was run. Section 5.3.4 and Figure 5.7 report what was measured.

Table 3.1: Injected defects and their predicted outcomes. Buffer overflow is listed but not injected.

Defect	Example in UECM	Result	Safe Rust
Memory safety			
Use-after-free	Handler uses a context after it was released	Crash/corruption	Prevented
Double-free	Release frees the same context twice	Crash/corruption	Prevented
Use-after-move	Request uses a context after it was moved	Empty identity	Prevented
Buffer overflow	No applicable scenario in the context store	n/a	Not injected
Concurrency			
Iterator invalidation	Bearer list changes while it is being iterated	Crash/corruption	Prevented
Data race	Two workers update the same field at once	Wrong value	Prevented
Unguarded shared mutation	Multiple workers mutate the same context	Wrong value	Prevented
Thread-unsafe handle	Non-atomic handle is shared between workers	Corrupted refcount	Prevented
Defects safe Rust does not automatically prevent			
Lock-order deadlock	Two handovers acquire locks in opposite orders	Hang	Not prevented
Lost update	Two workers overwrite each other's updates	Wrong value	Not prevented
Reference-cycle leak	Contexts keep references to each other	Memory leak	Not prevented
Integer overflow	TEID calculation exceeds the integer range	Wrong value	Not prevented
Worker failure	Worker fails while holding shard state	Wider outage	Not prevented
Double-free across FFI	C++ releases a Rust handle twice	Crash/corruption	Not prevented
Forgotten lock guard	Lock is not released after an early return	Hang	Not prevented

3.3 Research approach

The study runs experiments on a matched pair of versions. It builds that pair, plus the harness that drives them. The harness is the program that sends the load and records the times. Both are built from one frozen spec. Evidence comes from three separate methods: a load sweep contributes to RQ1 and RQ3, controlled fault injection answers RQ2, and

an FFI migration prototype answers RQ4. A weakness in one method does not weaken the others. How much the host can move the tail, for example, does not change the fault-injection result.

Each question asks how one outcome depends on one variable, the language. The language has to be the only planned gap between the two objects under test. That rules out the most common way to compare languages. In that form two programs written on their own are measured, and the gap is taken to be the language. Such a gap also carries the design, the libraries, and the skill of their authors. Building both sides from one spec removes that confound. Where a question can be answered with numbers, it uses numbers. Where it cannot, the answer is still by a fixed plan, in words. That is how this study treats the kind of guarantee in RQ2. RQ4 asks about the design cost of the application programming interface (API).

The closest prior work is the five vendor-hosted theses of Section 2.2. They appear in no citation database. They had to be found by searching each university's own archive by hand. The same searches found no Finnish thesis on Rust in a radio-access setting. They found no indexed paper mentioning Rust from the vendor hosting this work. The search returned no study that measures the tail of a Rust and a C++ version of the same stateful, multi-thread control-plane part under too much load.

Table 3.2 is the map for the rest of this chapter. Each row is one research question. The columns say which test answers it, what was built for that test, which data the test produces, and which number is reported.

RQ1 asks how long a call takes, and RQ3 asks how much processor time and memory the service uses. H1 is the hypothesis for both questions. It says the two languages are performance-equivalent below saturation, with any difference only in the far tail. One hypothesis then has to hold for the delay and for the resources together, which is why that wording is its main weakness. The two need not move together. Chapter 6 returns to that wording.

RQ2 records whether the compiler accepts the program and what the C++ sanitizers report. These results are independent of the machine. Most of RQ4 consists of sizes or counts, which are likewise machine-independent. Build time is measured with a stopwatch. A faster machine reduces both the Rust and C++ build times, so absolute times depend on the machine used. To control for this, the study reports the ratio of the two build times, measured sequentially on the same machine. Thus, the comparison is made within the same machine, rather than across machines.

Table 3.2: How each research question is answered. RQ1 and RQ3 use two methods. The first measures the full round trip, from deployment to completion. This shows the delay that a real deployment would experience below saturation. The second separates the core work from the handoff. This shows how much of the delay comes from the language itself. Past saturation, extra delay samples are written after the timed window.

RQ	Method	Artefact / data	Primary metric
RQ1	Load sweep to overload, interleaved repeats	Benchmark harness, latency histograms	median, p99/p99.9/p99.99, and maximum vs. offered load
RQ3	Same sweep, resource capture	`getrusage` samples per run	peak resident set size (RSS), CPU, throughput
RQ1, RQ3	Core called directly, then through the service	Matched microbenchmark pair	operation vs. cross-thread handoff
RQ2	Controlled fault injection	Defect catalogue, three C++ sanitisers, `rustc`, Loom/Shuttle, Miri	compile-time / runtime / undetected
RQ4	FFI migration prototype	C++ host + Rust C-ABI boundary	boundary size, `unsafe` scope, per-call cost

3.4 Methods per research question, in outline

RQ1 and RQ3 use an open-loop load sweep. The sweep runs from a light load to well past saturation. Open-loop means the generator keeps to its send schedule. It does not wait for a reply before sending the next request [38]. Four builds are compared. They take turns inside each repeat, so a noisy moment hits all four.

The time from send to reply mixes two costs. The core operation is the work the language does on the device record. The handoff passes that work to the worker thread and brings the answer back. A second timing measures those two costs apart. The round-trip spends most of its time on the handoff, so the split shows how much of the measured time the language's own work can account for.

A third timing measures the bounded queue and the reply handle on their own, without any `ShardCore` work in the path. The two languages build these pieces differently, so this test focuses only on the submit-and-wait synchronisation between the producers and the worker. Several producers compete for the same bounded queue, and each waits on a one-shot handle until the worker signals back. Together with the core-versus-handoff split above, this helps narrow down which part of the path may contribute to the overload

delay.

Past saturation, extra delay samples are written after the timed window. They record the wait without adding recorder work while the timer is running.

RQ2 is answered by controlled fault injection [39]. Each bug class is a small program of its own, written in both languages. The C++ side is built and run under AddressSanitizer, ThreadSanitizer, and UndefinedBehaviorSanitizer. Those three tools watch a running C++ program for memory bugs, thread bugs, and undefined behaviour. Undefined behaviour is a mistake the language does not define, so a running program may do anything. The safe-Rust side is compiled. Where it compiles, it is run with a time limit, and the run is stopped if it takes too long. No static analyser is used, on purpose. A static analyser reads the source without running it. The C++ columns report the tools this study ran. They do not claim that no other C++ tool could find more.

RQ4 is answered by a working prototype in the direction a real migration takes. A C++ host calls an unchanged Rust core across a C ABI. That thin layer converts between the C++ host and the Rust core. The per-call cost is measured on a real operation and not inferred from an empty crossing. Memory bugs that cross a language sit in that layer [40].

3.5 Fairness as a controlled variable

A language comparison is only as good as what it keeps the same. The spec is frozen before either version exists. Several choices are then matched on both sides, and never tuned to favour one language. Those choices are the thread model, the allocator and its arena count, the opt level, the hash function, the histogram method, and the load schedule. The opt level is how hard the compiler tries to make the code fast. An arena is a pre-sized pool the allocator draws memory from. Both sides use the same speed setting and the same pool count. Where the two languages could not be written the same way, the gap is written down and left in place.

The two sides are shown to match by comparing their logs, operation by operation. For a given seed they also run the operations in the same order. A seed is the starting number of that random order, so the same seed repeats the same sequence. The four builds take turns on the same pinned cores inside each repeat. A pinned core is a core the test is allowed to use, and no other work is. Running the same work one build after another keeps the processor's recent memory and recent branch guesses close. The cache is the small fast memory next to the processor. A branch guess is the processor's guess about which way a condition will go. The test does not empty the cache between builds. It also

does not first let the processor practise those guesses. The work and the cores stay fixed. It does not hold the CPU's recent memory and branch history fixed.

3.6 Statistical treatment

The statistical analysis plan was decided before the experiment was run. We first specified which comparisons would be tested. These comparisons could not be changed after seeing the results. In particular, a load could not be added to the statistical analysis because its results looked interesting in the load sweep.

For each planned comparison, we use an exact paired permutation test over 20 interleaved repeats. The test is *paired* because C++ and Rust are compared within the same repeat. This means that both languages are measured under the same conditions at approximately the same time. For example, if the system is temporarily slower during one repeat, that slowdown affects both measurements.

The permutation test asks whether the observed difference between C++ and Rust could have occurred by chance. To answer this, the test repeatedly swaps the C++ and Rust labels within the paired measurements. It then checks how often a difference as large as the observed difference occurs after these swaps. A small p-value means that a difference of this size would be unusual if there were no real difference between the languages.

For each comparison, we also calculate a bootstrap confidence interval [41]. The confidence interval shows the uncertainty around the measured difference. It indicates the range of differences that are reasonably compatible with the observed repeats.

Because several comparisons are tested, we apply the Holm correction [42]. Without a correction, testing many comparisons increases the chance that at least one comparison appears statistically significant simply by chance. The Holm correction accounts for the fact that several tests are performed.

There is a limit to how small the p-value can be with a fixed number of repeats. Before the Holm correction, this limit is 0.0000019. After the correction, it is 0.000027. We report these limits so that a reader can distinguish between a result showing no detectable difference and a result where the test does not have enough resolution to distinguish a smaller p-value. In this study, a *tie* means that the test found no statistically detectable difference between C++ and Rust.

Not every reported measurement is statistically tested. Some measurements are included only to describe the behaviour of the system.

Throughput at the selected load is not tested because the experiment itself fixes the request rate. The test sends requests at the selected rate, so this value is already determined by the experimental setup.

Jitter measures how much latency varies from one call to the next. It is reported as a descriptive measure because it is not part of the research questions.

Maximum latency is also not tested. The maximum is determined by a single call. One unusual or disturbed call can therefore determine the entire value. Pairing the repeats does not make this single-call maximum suitable for reliable statistical testing.

Pooled p99.99 latency is not tested either. This value is calculated from all individual calls combined into one distribution. Because the individual calls, rather than the repeats, are the observations, it cannot be used with the paired repeat-level statistical test.

Finally, latency percentiles are not statistically tested after saturation. At saturation, the sender can no longer maintain the planned request rate and starts to fall behind. This is called *schedule drift*. The measured latency then includes both the actual service time and the time spent waiting because the sender has fallen behind. Therefore, these measurements cannot be used as a fair estimate of the service's processing time and are reported only as descriptive results.

3.7 Reproducibility and research artefacts

The code, raw logs, and rebuild scripts live in https://github.com/VienThanh12/Master-Thesis. Access to that repository is restricted during review, and the same package is submitted with this thesis. A reader with either copy can rebuild the results without access to Nokia's private code. The versions are driven entirely by the public frozen spec. The benchmarking harness, the fault catalogue and its runner, and the FFI prototype are part of that package. Each plot of measured data is rebuilt by one script from the raw logs. So is each quoted number in the results. The script is `bench/refresh_thesis.sh`. That claim does not cover the hand-drawn schematics, which explain a concept rather than report a result. And running the same scripts over another host's campaign rebuilds the artefacts by the same procedure, not with the same values.

The logs that script reads are part of the package. Each campaign has its own script and its own macro prefix, so a figure from one cannot reach a sentence about another. Each run's host setup is recorded and stored beside the campaign data. Nothing in the results chapters is typed from those files by hand.

4 Design and Implementation

Nothing is built until the spec is frozen. It fixes the public API, the UE state machine, the data layout and the steps on it, the locks and shard size, and the thread model. Both versions follow it. The test then times the language, not a gap of design or of skill. The spec is itself a result of the study. It is the main guard against the author favouring one language while writing.

One thread model is used on both sides: shared-nothing and thread-per-shard. Shared-nothing means one thread never reads another thread's records. Thread-per-shard means each slice of those records has one owner thread. That layout is known to change tail latency by itself [43]. The layout is held fixed as a fairness control. A production gNB-CU-CP schedules UE work differently. The id space is split into shards. Each shard is owned by one worker thread. That thread alone reads or changes it. So no lock guards context state. Callers send a request to the owning shard's bounded queue. They wait on a one-shot reply handle. The queue blocks when it is full. That is on purpose. A drop rule would hide overload. This study needs overload to show up as latency. The only shared mutable state either version allows is one atomic counter. No control flow depends on its value. Figure 4.1 shows the data flow. There is one multi-producer, single-consumer (MPSC) queue per shard. Its worker drains it first in, first out (FIFO).

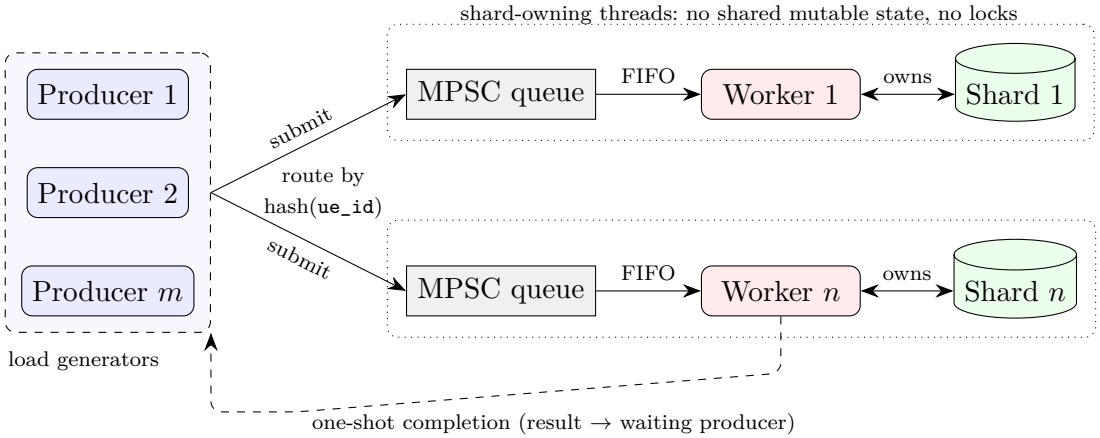


Figure 4.1: Shared-nothing, thread-per-shard architecture. Producers submit to a bounded MPSC queue per shard, and that shard's worker drains it.

Fairness is kept below the design as well as in it. Both builds link the same `jemalloc`. Both keep a system-allocator build as a control. Both compile at the same opt level. Both record latency through one high-dynamic-range (HDR) histogram set up the same

way [44]. Hashing is pinned on both sides. SplitMix64 assigns the shard [45]. FNV-1a, the Fowler–Noll–Vo hash [46], is used inside the map. C++'s default hash of an integer is often just the number itself. Rust's is strong on purpose. Left as they came, the two standard libraries would have been the measurement.

Where the languages cannot meet a need the same way, the gap is written into a forced-idiom log and then measured. Two of those gaps were candidates for the overload lead of Section 5.3.2. The first is the bounded queue, and Rust takes it from `sync_channel`. C++23 still has no such primitive, so C++ builds one from a mutex and two condition variables. The other is the reply handle, `std::promise` against a fresh `mpsc` channel per request. Tests of those primitives and a replacement reply handle were run on an earlier host. They do not explain the remaining lead on this one once the extra delay samples are written after the interval. A third carries the core-operation result of Section 5.3.1: the per-shard container is each language's own standard hash map. A fourth is the request encoding. C++ has no payload-carrying enumeration in this idiom, so a request becomes a tagged struct holding each field any operation might need, 160 bytes against Rust's 96. The net result is a Rust core 20% longer in code lines. Its branching complexity is 13% lower [47]. The Rust library contains no `unsafe` at all. In Rust, unsafe is the block where the compiler's checks are switched off.

Parity is tested and not assumed. Language-neutral vectors run in both versions. Their logs are diffed operation by operation. A cross-diff cannot tell "both correct" from "both wrong in the same way". A third version in Python acts as an independent oracle. An oracle is a third program, written on its own, whose answers the other two have to match. It has to reproduce all 223 hand-derived expectations before it may generate any of its own. A campaign of 9 000 generated operations found no mismatch. The C++ side runs under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer. The Rust side runs under Miri, an interpreter that checks `unsafe` Rust for undefined behaviour. A clean sanitiser result is bounded by the code that actually ran.

5 Experimental Setup and Evaluation

This chapter first describes the machines used in the experiment and the workloads that were run on them. It then presents the experimental results. Load comes first, as the service is swept from a light load to overload. The remaining research questions follow.

Several terms carry a specific meaning throughout this chapter.

Offered load is the rate at which the test sends requests to the service. For example, an offered load of 10,000 requests per second means that the test attempts to send 10,000 requests every second.

Saturation is reached when the service can no longer keep up with the offered load. Past this point, a higher requested rate does not raise the rate at which the service finishes work, because work begins to accumulate or the sender begins to fall behind.

Median latency is the latency at the middle of the measured observations. Half of the calls are faster than the median and half are slower.

Tail latency is the slow end of the latency distribution. It describes unusually slow calls rather than the typical call. Percentiles such as p99 and p99.99 name this part of the distribution.

5.1 Testbed setup

Some numbers in this chapter come from the tools and the code. Other numbers depend on the machine they ran on. The reported campaign comes from one bare-metal host, a single-socket AMD EPYC 7443P. Bare metal means the program runs on the machine itself, with no virtual machine in between. Each core runs one thread. Extra threads on a core are off. The host runs Ubuntu 24.04.4 LTS on kernel 6.17.0-23-generic. Linux can raise and lower each core's clock as load and heat change. The clock is held fixed. That clock setting is performance, and boost, a further rise of that clock, is off. The test may use only cores 2–17. There is no hypervisor. A hypervisor is the program that would run a virtual machine. The program runs on the host itself, not inside a virtual machine. On those cores the kernel turns off its timer tick (`nohz_full`). If left on, those ticks would

show up in the latency numbers. The study does not use `isolcpus`. That flag also stops the kernel sharing work across the allowed cores. Each test thread is placed on one core.

The compiler and build tools used in the experiment are not taken directly from the host system. Instead, both the C++ and Rust programs are built and executed inside the same container image. The image is based on Rocky Linux 8.10 (Green Obsidian) and provides `g++ 15.2.1` and `rustc 1.97.1`. This ensures that the build environment is controlled and reproducible rather than depending on the software installed on the host machine. The image specifies the major version of the GNU Compiler Collection (GCC) toolset, but not its exact point release. As a result, the compiler version used in this experiment is `g++ 15.2.1`. The use of a container does not completely isolate the experiment from the host system. In particular, a container uses the host's Linux kernel.

Until late in the study, the load sweep was performed inside a Kernel-based Virtual Machine (KVM) running on an Intel(R) Xeon(R) CPU E5-2680 v4. The CPU core controls described above were not applied during this earlier sweep. The experimental notes record the hypervisor as kvm, the guest operating system as Rocky Linux 8.10 (Green Obsidian), and the guest kernel as 7.0.2-1.el8.elrepo.x86_64. Here, the guest refers to the virtual machine itself, while userland refers to the programs and libraries that run on top of the operating-system kernel. The earlier sweep showed substantial variation in the measurements. For one variant, the 95% bootstrap confidence interval for p99 latency had a width of 1480 μs. Across the four variants, the difference between the smallest and largest p99 values was only 293 μs. In other words, the uncertainty in the measurement of one variant was larger than the difference observed between all four variants. This means that, although the four variants had different measured p99 values, those differences were small relative to the variability of the experiment. The results should therefore be interpreted with caution: the observed differences between the variants are small compared with the uncertainty in the measurements. We nevertheless retain this earlier sweep because it used the same experimental design as the main experiment. This makes it useful for a side-by-side comparison. The purpose of this comparison is to examine whether the observed findings are consistent across different environments and therefore less likely to be specific to one particular machine or environment.

A third sweep was performed on a second bare-metal machine with an AMD EPYC 7443P processor. This sweep changed several experimental choices at the same time: the shard count, the set of offered loads, the number of repetitions, and the tools used. It therefore provides a broader robustness check. Specifically, it tests whether the findings remain similar when several experimental choices are changed together. It does not allow us to determine the effect of any one of these changes separately.

Appendix A provides a field-by-field description of the three hosts and their corresponding experimental settings.

5.2 Workload and measurements

The work is made for the test. The steps it takes come from the 3GPP rules [1, 4]. A driver that always sends the same steps walks each UE through the connection lifecycle those rules define. The *signal* work first connects a set of UEs, then sends the same signal call again and again. That picks out the common path. The *mixed* work walks each UE through context creation, setup, signal, bearer setup, reconfiguration or handover, and release. A fixed random order makes both sides see the same counts of each call kind for a given seed. That is also a check that the two sides did the same work. Figure 5.1 shows the two loads and the steps the mixed one walks.

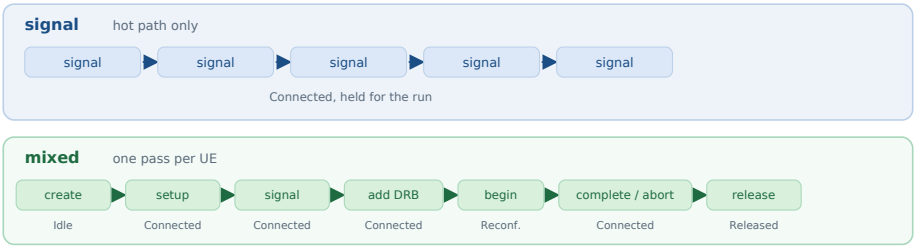


Figure 5.1: The two workloads, and the lifecycle the mixed one walks. Signal repeats the common path, and mixed walks each UE through the procedure sequence drawn below.

The test follows the 3GPP rules, which do not specify a particular mix of call types. In a real mobile cell, the handover rate depends on factors such as the location of the site and the time of day. There is no single publicly defined handover rate. Therefore, the call mix used in this study may differ from that of a real cell. However, the same mix is applied to all variants, so any difference in the mix affects all variants in the same way. It may change the absolute results, but it does not make the comparison between variants unfair.

The sender uses an open-loop approach, meaning that it continues to send requests according to the planned schedule even when the service becomes slow. If the producer falls behind this schedule, the test harness adds samples to the delay histogram for requests that should have been sent during the missed period. This follows the coordinated-omission correction approach used in HdrHistogram [38, 44]. In this study, this process is referred to as *histogram fill*, with up to 10 000 synthetic samples added for each completed call. These additional samples represent delays that could otherwise be missed when the sender falls behind. The term *histogram fill* is specific to this test harness and is not a standard

benchmarking term.

Each load level is tested 20 times. The four language and allocator variants are interleaved within each run to reduce the effect of changes in the test environment. Each reported value is the median of the measurements, together with a bootstrap 95% confidence band [41, 48].

For the resource-usage plots, the study uses a fixed load of 56 kops/s (56,000 operations per second). This value was selected independently rather than taken from the load sweep. The initial rule was to use the highest load at which the p99 latency remained below 5 ms. However, on the virtual machine, this value was strongly affected by noise from the host system. On a quiet host, the load can be increased until the service is no longer able to keep up. Therefore, this measurement is used only as a check.

The lowest completed rate among the four variants is 392 kops/s, which occurs when the offered load is also 392 kops/s.

5.3 Results

The safe-code results and most migration results do not depend on the host used. However, the speed and RSS results do. Unless otherwise specified in the figure captions, all measurements presented below were obtained on the host described in Section 5.1.

5.3.1 Below saturation (RQ1, RQ3)

This part compares the two languages while the service is still able to keep up with the offered workload. At the chosen load of 56 kops/s, all four builds remain sustainable. Each can process the incoming requests without falling behind.

At this load, the four median latencies differ by at most 1.6 μs, which is 2.2%. A paired test still finds a gap of 1.5 μs between the languages. Rust has slightly lower median latency than C++ under both memory allocators. Figure 5.2 shows the load sweep behind that number.

The same analysis extends to the higher tail percentiles. For one variant, the widest 95% band on its own p99 spans 0.8 μs. The four variants' p99 values differ by 1.8 μs. The uncertainty on one variant is therefore smaller than the gap between them. On the virtual machine the same two sizes were 1480 μs against 293 μs, the other way around.

Both p99 comparisons and both p99.9 comparisons remain significant after correction for

multiple comparisons, 4 in all. That matches the median: the language gap is detectable at the typical call and in the upper tail.

Table 5.1 reports p99.9 and the maximum at the chosen load. The maximum is kept out of the tested family, as the pooled p99.99 is.

Table 5.1: How long one call takes at the chosen load, 56 kops/s, in microseconds. The median over 20 repeats sits above its bootstrap 95% interval. †The p99.99 column pools every repeat, so it has no interval. The maximum has an interval across repeats but stays outside the tested set, because one disturbed call can set it.

			(µs)		
Variant	p50	p99	p99.9	p99.99†	max
C++ (system alloc)	73.9	76.9	81.0	110.0	226.4
	[73.9, 74.0]	[76.4, 77.2]	[80.7, 82.0]		[221.1, 239.3]
C++ (jemalloc)	73.8	77.1	80.2	143.0	227.8
	[73.7, 73.9]	[77.0, 77.1]	[79.9, 80.9]		[217.2, 244.4]
Rust (system alloc)	72.7	75.3	77.1	87.9	100.3
	[72.6, 72.7]	[74.9, 75.6]	[76.9, 77.9]		[97.4, 160.8]
Rust (jemalloc)	72.3	75.7	77.8	81.3	104.4
	[72.2, 72.3]	[75.6, 75.8]	[77.6, 77.9]		[94.1, 141.8]

The far tail shows the same separation between the languages. The runs' histograms are pooled, which gives 3 360 000 to 3 360 063 calls per variant. The two Rust variants sit below both C++ variants. The widest pair differs by 62 μs. The slowest Rust variant still sits 22 μs below the fastest C++ one. A second bare-metal host at another shard count agrees, running 147–228 μs for the C++ variants against 88–115 μs for the Rust ones, and the ranges do not meet. A second limit does not come from the host. No host would remove it. An end-to-end round-trip spends only 0.250% of its time on the work the language does. The rest is passing the request to another thread, and that time is charged to the operating system. When the work inside the language is this small, time spent crossing from thread to thread dominates [49]. Figure 5.3 draws that split. This sweep cannot bound a language effect either way, which Figure 5.4 sets beside what the sweep can still show. Timed on its own, the core call is 1.6× faster in Rust. The test does not trace the cause. The stock hash map is one candidate. Compiler code generation may also own part of the gap.

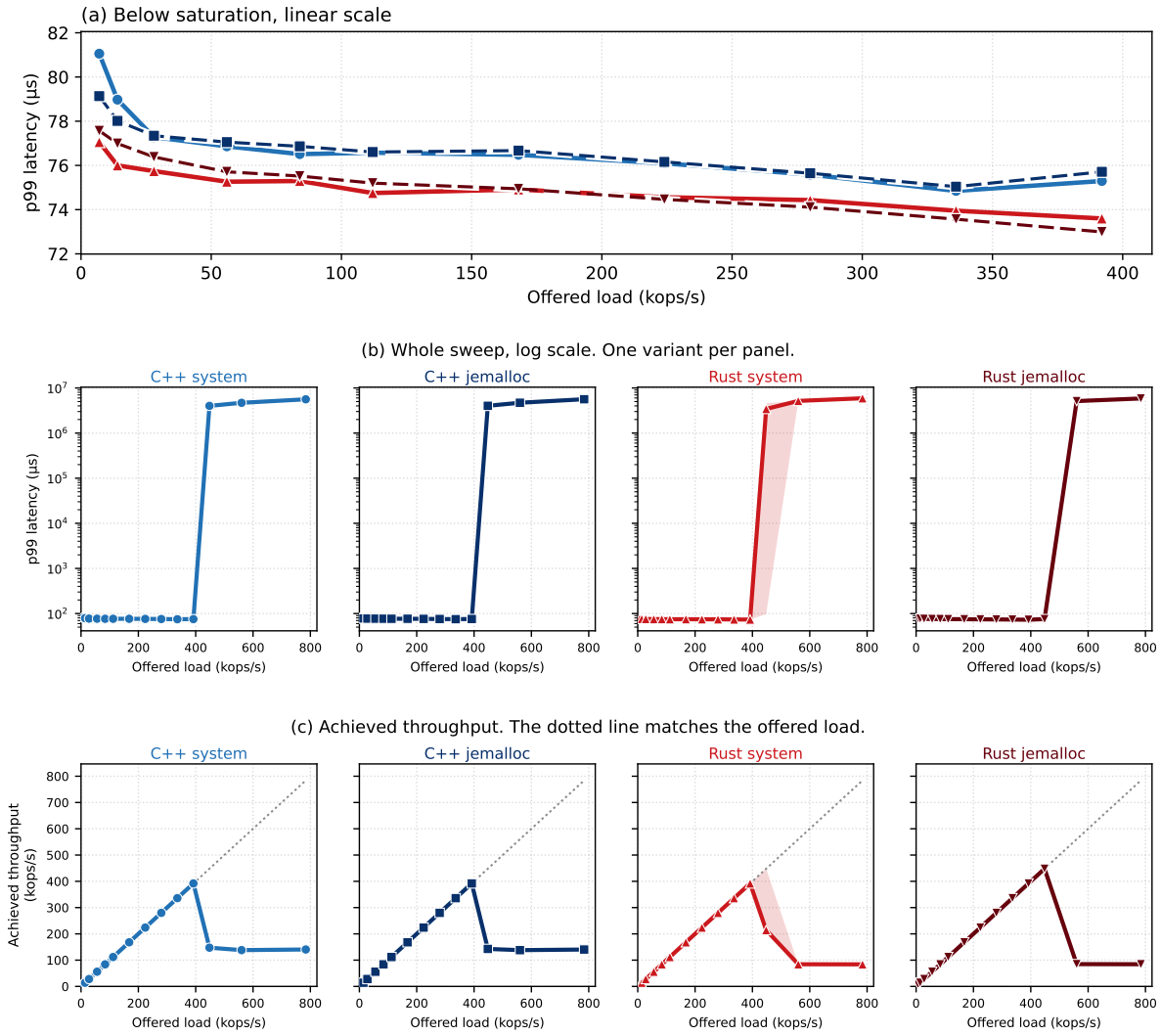


Figure 5.2: Panel (a) is p99 below saturation, on one linear scale, through 392 kops/s. The four variants sit within a few microseconds, so they are drawn together. In (b) and (c) each variant has its own panel. Panel (b) is the tail on a log scale across the whole sweep. Panel (c) is achieved throughput, and the dotted line matches the offered load. Lines are medians over 20 repeats, and the bands in (b) and (c) are bootstrap 95% confidence intervals.

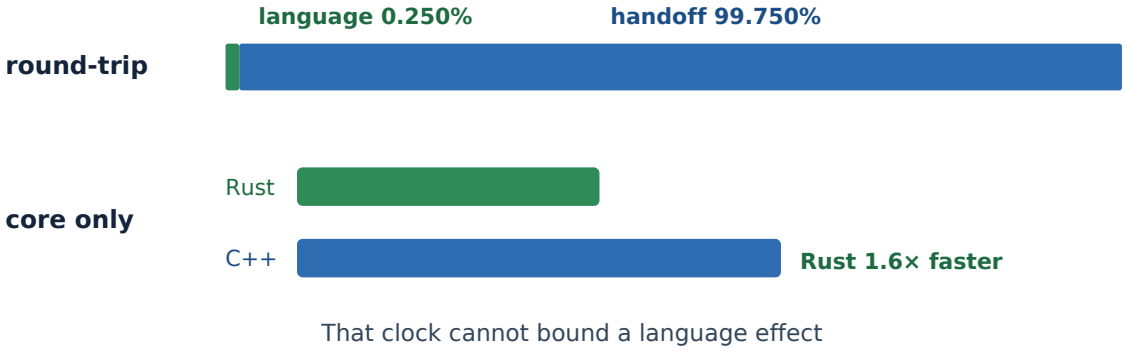


Figure 5.3: Where an end-to-end round-trip's clock goes. The language share is 0.250%. A clock that spends the rest on handoff cannot bound a language effect.

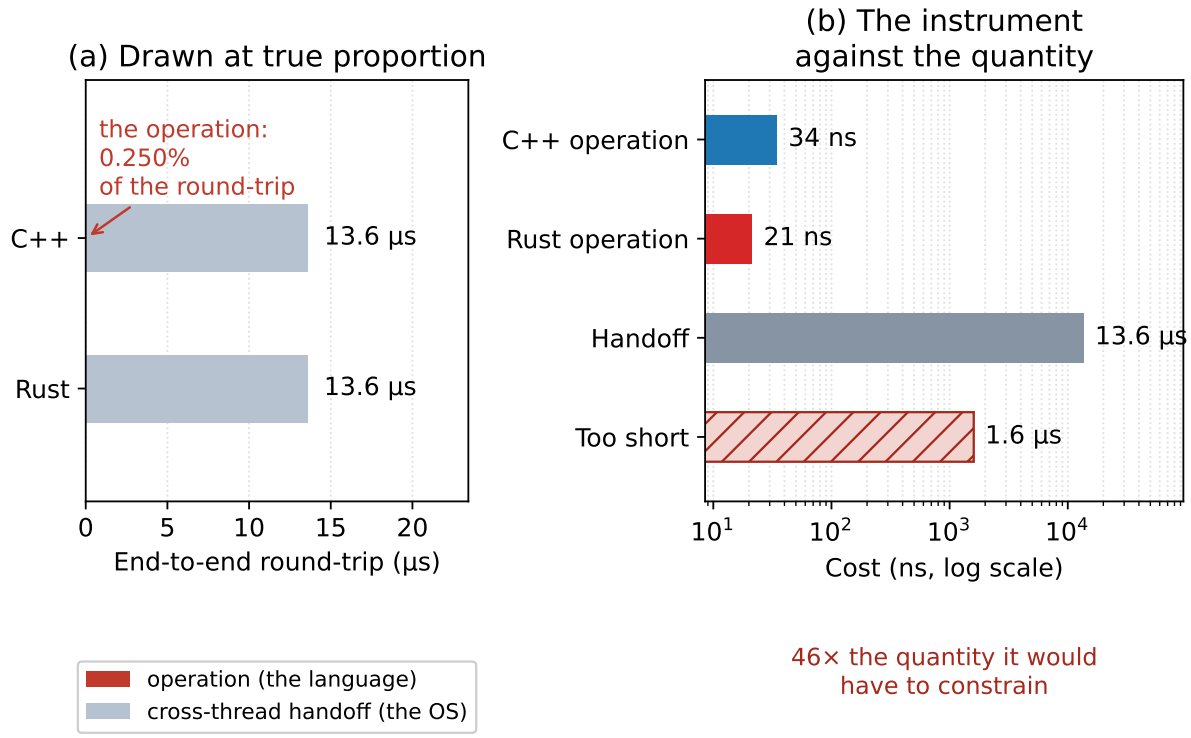


Figure 5.4: Where an end-to-end round-trip's cost goes. **(a)** The round-trip at true proportion. **(b)** Both operation measurements are too short for that clock to separate, by a factor of 46.

5.3.2 Past saturation (RQ1)

This part explains what happens when the test sends more work than the service can process. Every request has a planned send time. When the service is slow, the sender is still waiting after that time has passed. If the log kept only the calls that finished, the wait would not show up as delay. The harness writes extra delay samples for the requests that should have been sent during the wait. Those samples are written after the timed window, so they do not add recorder work while the timer is running. The run uses the same grid and is repeated for 1120 runs.

Figure 5.5 maps the representative load, the shared tracking load, the load where C++ first falls behind, and the overload point.

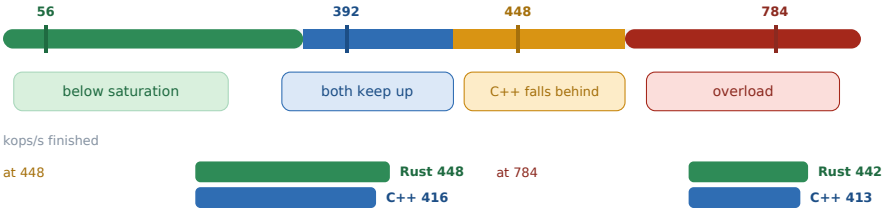


Figure 5.5: The figure shows four load points, ranging from light load to overload. These points are placed along the offered-load axis for easy reading. Their positions are not to scale. The tick marks show 56, the shared tracking load 392, the load where C++ first falls behind at 448, and the overload point at 784 kops/s. The bars show finished throughput, with extra delay samples written after the timed window.

At 448 kops/s, C++ serves 416 kops/s and Rust serves 448. Rust still keeps up to 448 kops/s, while C++ keeps up only to 392.

At the highest load, Rust is still ahead on both allocators. On the system allocator, C++ finishes 0.95 of Rust's throughput, 412 against 434 kops/s, at $p < 0.001$. Under `jemalloc`, C++ finishes 0.92 of Rust, 413 against 450 kops/s, at $p < 0.001$. These comparisons were not in the tests chosen before the experiment.

Once throughput has stopped rising, CPU per finished call is 1.03, with C++ the higher one.

The remaining Rust lead is not from the core call. Each sender waits for its reply before sending the next request. Past saturation, the rate of finished calls is therefore set by the round-trip time. The core call is at most 0.250% of that trip, far too little to explain a lead of this size. The rest of the trip is handing requests between threads.

Most of the CPU a call uses at the top load, 57 to 63%, is kernel time. The operating

system spends it putting threads to sleep and waking them. C++ uses more kernel time per call than Rust at 14 of the 14 loads on both allocators, and 1.12 times as much at the top load.

Where the remaining lead sits. One further host, with the same processor, then swapped the hash map and the queue. Rust keeps map entries in one flat table. C++ gives each entry its own node. Putting an open-addressed map in C++ left the lead in place. Rust was ahead in 10 of 10 paired repeats.

The original Rust queue is `sync_channel`. The swap gave Rust the C++ locking design instead. Only one thread may change the queue at a time. A sender waits for a free slot, and a worker waits for a request. That check is not the language comparison. It asks which part carries the remaining lead. Rust with that lock lost the lead. It fell behind the original Rust channel in 9 of 10 repeats.

C++ then kept the same lock and changed only the buffer, from a growable buffer to a fixed ring. Throughput moved by only 0.4%. The lead therefore comes from how the queue is locked, not from how it stores requests. The lock adds kernel time to each handoff. It does not add extra thread sleeps. All four builds switch threads 2.8 times per call at 56 kops/s.

Delay with several requests in flight. After the system becomes saturated, the sender starts to fall behind. The reported delay then includes two things: the service time and the time the sender is behind schedule. Therefore, this delay does not by itself tell us which service is faster.

We measure the actual work separately. Several requests can be in flight at the same time. Each request keeps its planned arrival time, and we stop measuring when the worker finishes it. This experiment was run on a different machine, using processor AMD EPYC 7443P 24-Core Processor. It is not part of the tested system family reported in the main campaign.

The offered load is 3920 kops/s. We chose this load as the first measured load where every variant completes fewer than 99% of its planned requests within their arrival windows. We chose this rule before looking at which variant performed better.

Table 5.2 shows the median results for repeated signalling and mixed lifecycle. With `jemalloc`, Rust has the lower median in every repetition for both workloads. With the system allocator, C++ has the lower median throughout.

The difference comes from the time spent waiting to enter the queue, not from the time spent doing the work.

Table 5.2: Delay with several requests in flight on a separate host at 3920 kops/s. The median is the time spent waiting before entering the queue. In-service time is measured after queue entry until completion. This experiment is outside the tested system family reported in the main campaign.

	Signal		Mixed	
	system	`jemalloc`	system	`jemalloc`
C++ median (ms)	200.7	121.9	199.8	120.4
Rust median (ms)	265.5	53.8	279.1	73.9
Rust lower	0/20	20/20	0/20	20/20
C++ in service (µs)	14.5	21.2	13.2	18.6
Rust in service (µs)	4.0	3.3	4.1	3.4

5.3.3 Resource use (RQ3)

The four variants have similar median latencies, but they do not use the same amount of RAM or CPU. The resource results show different patterns for memory and CPU, so we discuss them separately.

Memory usage. Peak resident set size (RSS) shows two distinct effects: a *language effect* and an *allocator effect*. The language effect measures the additional peak RAM used by Rust compared with C++, while the allocator effect measures the additional peak RAM used by `jemalloc` compared with the system allocator. Table 5.3 reports both effects on all three hosts, and Figure 5.6 shows the results for the reported host at 56 kops/s.

The language effect is consistent across all three hosts: Rust uses more peak RAM than C++, and the difference remains within a few MiB. However, its exact size varies by host. In contrast, the allocator effect varies much more between hosts. On the virtual machine, for example, the allocator effect is about 13 times larger than the language effect. Thus, the relative size of the two effects is a property of the host environment, rather than a general property of Rust or `jemalloc`.

Table 5.3: Extra peak RAM on each host, in MiB. The language effect is the difference in peak RSS between Rust and C++, averaged across the two allocators. The allocator effect is the difference in peak RSS between `jemalloc` and the system allocator, averaged across the two languages. Each cell reports the range of these differences computed from the medians of the repeated runs. The reported host is measured at 56 kops/s. The other two hosts are measured at their respective steady-state loads.

	Reported host	Virtual machine	Second bare-metal host
Language effect	1.6–2.5	1.8–3.5	0.7–1.8
Allocator effect	1.5–2.3	32.5–34.3	1.9–3.2

The second bare-metal host provides more detail about the source of the additional Rust RAM. Table A.4 separates the process memory into allocator metadata, live application allocation, and allocator-resident memory. Compared with C++, Rust uses +2.07 MiB more arena metadata, but −0.16 MiB less live application allocation and −1.92 MiB less allocator-resident memory. Despite using less live application and allocator-resident memory, the Rust process still has a peak RSS that is +1.74 MiB higher.

Therefore, the higher Rust RSS is not caused by Rust holding more live application data. Instead, it is mainly a fixed process-level cost.

CPU usage. CPU usage does not follow the memory pattern. At 56 kops/s, Rust uses at most 6.2 percentage points less CPU than C++. At 784 kops/s that ratio is 1.03, with C++ the higher one. A throughput lead remains, and Section 5.3.2 traces it to how the queue is locked.

What the hardware counters tell us. The hardware-counter measurements on the second bare-metal host cover only an isolated call. There is no queue, no offered load, and therefore no competition between requests for shared resources. The measurements show how much CPU work is needed to process one request when it runs by itself.

For this isolated call, Rust retires 11% fewer instructions and completes in 15% fewer CPU cycles than C++. It also has 29% more cache misses per thousand instructions. Thus, Rust completes the isolated call faster even though its cache-miss rate is higher.

These measurements show why Rust is faster than C++ when a single call is processed in isolation. However, the overload experiment is different: many requests arrive at the same time and must share the queue and its lock. Some requests therefore spend time waiting for the queue or for other threads to release the lock. This waiting does not occur in the isolated-core experiment and is not captured by the hardware-counter measurements. The overload experiment in Section 5.3.2 showed that the additional overhead comes from waiting for the queue lock and from the kernel time required for each handoff between threads. Therefore, the hardware-counter results explain the performance of an isolated call, but they do not explain the additional delays that appear when the system is overloaded.

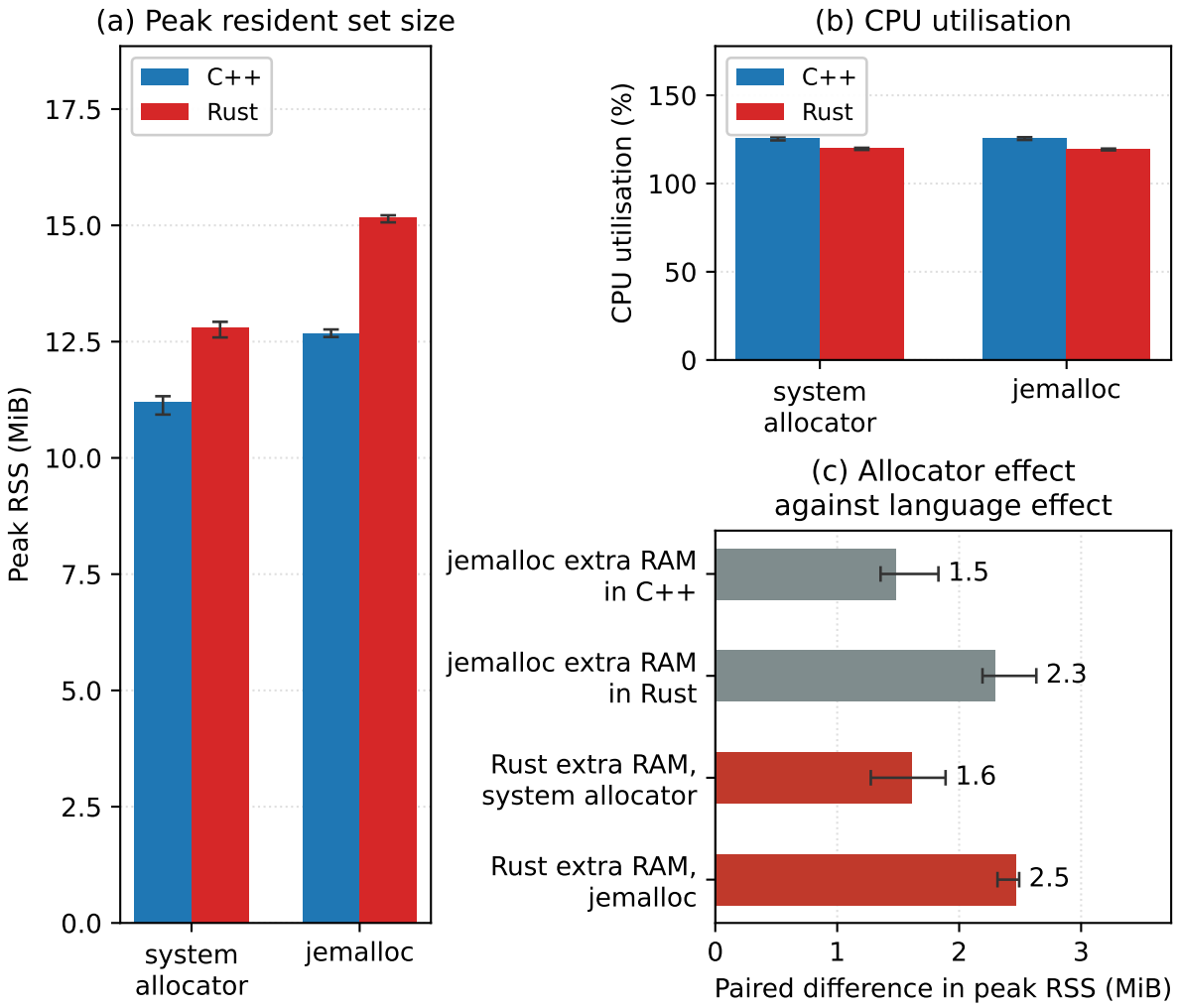


Figure 5.6: Resource use at 56 kops/s, grouped by allocator. Bars are medians over 20 repeats with bootstrap 95% intervals, and CPU in (b) is summed across threads.

5.3.4 Safety via fault injection (RQ2)

Safe code is evaluated by injecting 14 defect classes into the implementations. Seven of these defects fall within the safety guarantees that Rust is intended to provide, while the other seven represent defects that safe Rust is not expected to prevent. The first seven defects are prevented by the Rust compiler at compile time. In C++, the same defective code is allowed to compile and run. The defect will only become visible if the program actually runs the part of the code where the defect exists. Different defects require different detection methods, so no single sanitiser can detect all of them.

Use-after-move is an important example because it is prevented by Rust even though no sanitiser reports it. A moved-from C++ object remains valid, but its state is unspecified, so the program may continue running with the expected identity missing. There is therefore

no invalid memory access for a sanitiser to detect, even though the application behaviour is incorrect (Section 3.2).

The seven boundary classes also compile in safe Rust. Rust does not rule them out at compile time, so the faulty behaviour can still happen when the program runs. Some can nevertheless be detected by specialised tools. LeakSanitizer detects the reference cycle because the resulting memory leak is permitted by Rust's ownership model [50, 51]. UndefinedBehaviorSanitizer detects the integer overflow in C++, while Rust defines the same operation to wrap when overflow checks are disabled. ThreadSanitizer detects the lock-order inversion in 3 of 12 runs, although this result varies between runs. The tool records lock-order relationships only when the relevant locks are successfully acquired. If a thread becomes stuck while waiting for the first lock, the other lock may never be acquired. As a result, ThreadSanitizer cannot observe the complete sequence of lock acquisitions needed to identify the deadlock, so it may not report the problem.

The double-free inside `unsafe` Rust is not a separate defect class, but the same double-free error occurring in code where Rust's normal safety checks have been bypassed. The C++ implementation contains the corresponding error, so the difference between the two implementations is only in whether the compiler is able to prevent it. Miri detects the Rust double-free, but it cannot execute the C++ host code behind the foreign-function boundary.

Some concurrency defects cannot be found directly by the sanitiser tools. To investigate these cases, this study used Loom and Shuttle [52]. They systematically explore different possible orders in which threads can execute. These tools operate on small models rather than the full application. Each defect had to be converted into the tool's model and simplified to the smallest example that still reproduced the problem.

Rust's Resource Acquisition Is Initialization (RAII) prevents one common concurrency defect, forgetting to unlock a mutex. The lock is automatically released when its guard is dropped. RAII does not eliminate all lock-related defects. The two forgotten-guard cases in the study still caused the programs to hang, and neither tool reported these hangs. Figure 5.7 draws the whole list as an outcome matrix.

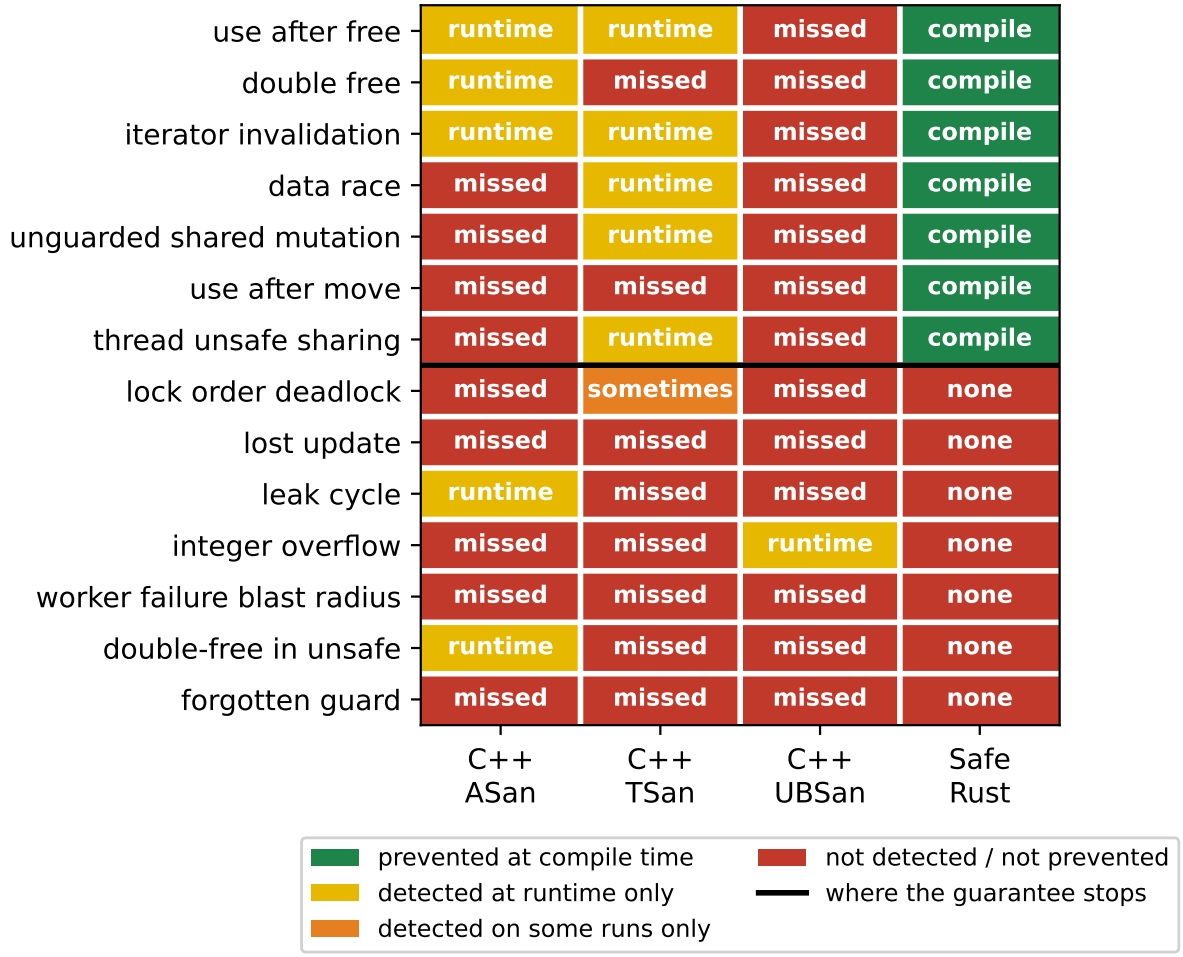


Figure 5.7: The fault-injection outcomes as a matrix, generated from the taxonomy log.

5.3.5 Migration (RQ4)

The C-ABI boundary between the C++ host and the Rust core is small. It consists of 11 C entry points implemented across 231 lines of Rust. All raw-pointer operations are isolated in 9 functions, leaving the main Rust core free of `unsafe` code.

We observe no measurable runtime overhead from crossing this boundary when comparing the same signal call made directly with the call made through the C-ABI. Thus, any per-call overhead introduced by the boundary is bounded by 10.91% of the call cost in our measurements. Figure 5.8 shows the structure of this boundary.

We also measured three practical costs of adopting Rust beyond the FFI boundary: build time, compiler cost for the project's own source, and shipped binary size. Third-party code accounts for 54% of the CPU time in a clean Rust build. When we consider only the project's own source, the Rust build requires 0.8 times as much CPU as the corresponding `g++` build.

The observed 6.8× binary-size difference is also strongly affected by the linkage policy. When the same linkage policy is used for both languages, the size difference changes and can even reverse. Finally, the Rust code used in this comparison contains no third-party crates and no `unsafe` sites (0 crates and 0 `unsafe` sites). These controls make the comparison more directly attributable to the language and build configuration rather than to dependencies or unsafe code.

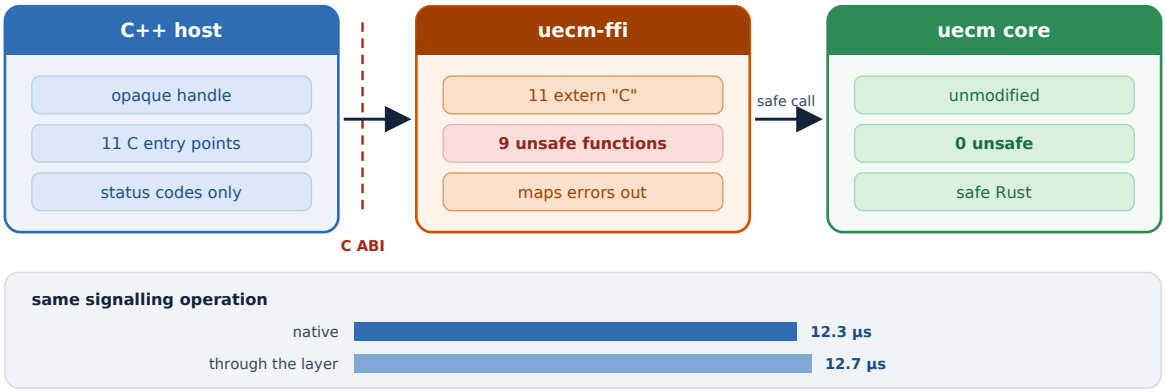


Figure 5.8: The layer where C++ calls the Rust service. The same signalling operation costs 12.3 µs directly and 12.7 µs through that layer, a paired difference bounded at 10.91% of that call. A do-nothing crossing costs 2.54 ns and bounds only itself.

6 Discussion

6.1 Summary of findings

At 56 kops/s, the variants show similar performance: their median latency differs by only 1.6 μs, while the pooled far tail favours Rust. Because only 0.250% of the end-to-end round trip is spent on language-specific work, the overall measurement cannot be used to establish a language-level performance effect.

When the same core is timed separately, 1.6× higher performance is observed in Rust. At higher loads, however, Rust still completes more calls. This advantage appears to come from the inter-thread handoff rather than from the language itself. When Rust uses the C++ queue on the host that swapped it, the advantage disappears.

After the system reaches saturation, the campaign delay mainly reflects scheduling drift and should not be used to compare the services. Instead, timing the work with several requests in flight provides a more meaningful comparison. As discussed in Section 5.3.2, the performance order in this case depends on the allocator.

For RQ2, the outcome is fully determined by the fourteen-class catalogue defined in advance and is consistent across hosts. All seven classes of errors that safe Rust is designed to reject result in compile-time errors. The remaining seven compile. They fail at runtime in both Rust and C++.

RQ3 shows that Rust has higher peak RSS under both allocators. The difference is 1.6–2.5 MiB on the campaign host, 1.8–3.5 MiB on the guest, and 0.7–1.8 MiB on the second bare-metal host. Below saturation, Rust uses slightly less CPU. After correcting for high-load effects, however, the throughput measurements do not support a general performance ranking between the languages.

RQ4 finds that the FFI layer contributes little overhead: its per-call cost is bounded by 10.91% of the corresponding call. Other factors, such as API design and learning effort, remain literature-backed adoption costs, but they are outside the scope of what this study measures directly.

6.2 Discussion on the research questions

RQ1 has two answers, depending on the workload level. At 56 kops/s, the campaign detects differences in the median, p99, and p99.9 latencies, while the pooled p99.99 latency favours Rust. However, the end-to-end measurements cannot be used to isolate a language effect because the core operation accounts for only 0.250% of the measured time.

After the system reaches saturation, the campaign delay mainly reflects scheduling drift rather than service performance. Additional delay samples are recorded after the timed window, so they do not add recording overhead while the timer is running. Therefore, these measurements should not be used to rank the services.

A more meaningful comparison comes from timing the actual work while several requests are in flight. Under this measurement, the performance order depends on the allocator. The measured delay includes the time before queue entry, the time waiting for the signal, and the overhead of the mixed lifecycle.

After saturation, Rust still completes more calls with both allocators. This advantage appears to come from the handoff between threads, where C++ spends more kernel time per call. On the host where the queue was swapped, replacing the Rust queue with the C++ queue removed the advantage. This suggests that the difference is caused by the queue and thread-handoff mechanism rather than by the Rust language itself.

The lower Rust p99 reported in the earlier run does not provide evidence about tail performance for RQ1. That experiment intentionally excludes the stalls, so it cannot be used as a tail-latency result. The overload throughput measurement is instead a comparison of completed work. Its campaign-clock percentiles mainly reflect scheduling drift.

The closest prior work reports a similar performance degradation above a threshold rate and attributes it to stackless `async/await` [32]. That explanation does not apply here because neither implementation uses an asynchronous runtime. Similarly, changing Rust's reply handle on an earlier host does not explain the remaining performance advantage observed here.

The throughput advantage should therefore not be interpreted as showing that one service has a higher maximum rate in all conditions. The original prediction attributed the difference to bounds and overflow checking [53, 54]. The results do not support that explanation. At nominal load, however, the median results are still consistent with previous evidence that Rust's safety checks are largely absorbed by modern out-of-order CPUs [12, 35].

For safe Rust code, some defects result in compilation errors and therefore never produce a program that can run. The seven classes of defects that Rust does not prevent are therefore part of the result, rather than an additional warning. A compiler that rejects a program rules out a defect for every run of that program. A sanitiser can rule out a defect only for the executions covered by the sanitiser test. In other words, sanitisation improves the coverage of the tested executions, but it does not reduce the overall range of possible defects.

Figure 6.1 shows where the guarantees provided by each approach stop. Rust provides strong guarantees about memory safety and aliasing, but it does not guarantee correct lock ordering. For example, a `Mutex` guard whose lifetime is tracked by the borrow checker does not guarantee that two threads acquire multiple locks in the same order. Static deadlock detection in Rust remains an open research problem [55].

A lost update is similarly outside the protection provided by the borrow checker and is not necessarily detected by ThreadSanitizer. When every access is correctly synchronised, there is no unsynchronised access for the tool to report. A field study of 105 real concurrency bugs found that about one third of the non-deadlock cases violated the intended ordering of operations rather than the locking rules themselves [56]. Rust therefore removes some classes of concurrency defects but does not remove defects that arise from incorrect concurrent logic.

For this reason, adopting Rust should not lead to the assumption that “memory-safe” means “concurrency-safe.” Such an assumption could result in insufficient review effort. In the corpus studied by Xu et al., all but one published Rust Common Vulnerabilities and Exposures (CVE) entry related to memory safety involved `unsafe` code. The exception was a compiler bug [57].

This is why keeping the core free of `unsafe` code is important. It also explains why the spread of `unsafe` code through dependent crates should be treated as a review concern [58].

One case shows that testing can also fail to cover a defect. A specification gap allowed a call to be sent outside the service’s running window. The call passed every test, every parity vector, and all three sanitisers because the affected path was never executed. On the Rust side, the failure became detectable because of a simple bounds check, not because of a compile-time proof. This suggests that the useful engineering principle is “checked by default,” rather than “ruled out at compile time.”

The root cause of the problem also belongs to the testing method. A rule without a corresponding test or enforcement mechanism leaves each implementation vulnerable to

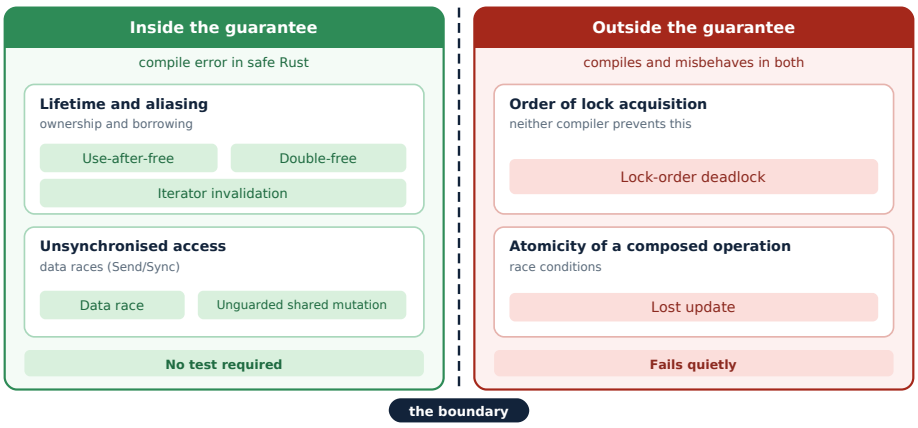


Figure 6.1: Where Rust's compile-time guarantee stops, with defect classes grouped by the mechanism that rejects them.

its own failure modes.

RQ3 gives different results depending on the resource being measured. The RAM result is consistent across both allocators and on a second host using a newer compiler. Peak RSS measures the maximum memory used during the entire run, so the jitter that affects p99 latency does not directly affect this metric. After accounting for the allocator's own contribution, the two implementations use memory at roughly the same scale on both bare-metal hosts. On the guest system, however, the allocator effect is an order of magnitude larger. Thus, the language effect remains visible, but its size depends strongly on the environment and allocator.

The results from the quieter host also show that the memory cost is largely explained by the allocator itself. One related study reports the opposite result, with Rust using less memory and the difference increasing with concurrency [29]. These results can be reconciled if the implementations are viewed as using different concurrency runtimes rather than as direct comparisons of Rust and C++.

In that study, the C++ implementation uses Boost.Fibers, which allocates a stack for each fiber. The Rust implementation uses Tokio, where a task is represented by a small future stored on the heap. Therefore, the measured difference is partly a difference between two concurrency runtimes.

Two additional studies are consistent with the findings reported here [30, 59]. Below saturation, Rust uses slightly less CPU. After saturation, the CPU ratio reaches 1.03. This difference cannot be directly attributed to Rust's safe-code model. Nevertheless, Rust still maintains a throughput advantage.

RQ4 shows that the boundary between the two languages adds relatively little overhead.

A C++ host can call an unchanged Rust core through a thin C-ABI layer, and the overhead of a real signal call is bounded by 10.91% of the total call time. Previous work identifies design and learning as important human costs [60, 61], but this study does not measure their magnitude.

Keeping `unsafe` code inside the interface layer limits the amount of code that requires additional review, although it does not eliminate the possibility of memory corruption. The host and the Rust component share the same address space [62, 63]. The build-size and footprint results point in the same direction: organisations concerned about these factors should examine their dependency and crate budgets before attributing the cost to the compiler.

The main area where C++ has a clear advantage is the edit-and-compile cycle for changes that affect only a single source file. The 6.8× size result also shows that the relative size depends on the chosen linkage policy. The ordering changes when the same policy is applied to both implementations.

6.3 Revisiting the hypotheses

Section 1.4 defined three hypotheses before any experiments were run. Revisiting them after the experiments makes clear which parts are supported and which are not.

H1 H1 is not accepted as written. No equivalence margin was defined, so a statistically significant but small median difference cannot be used to establish performance equivalence. At 56 kops/s, the paired test detects a difference of up to 1.5 μs in the median and also detects differences at higher percentiles. The pooled far-tail result favours Rust, while the isolated core favours Rust.

After saturation, the campaign delay mainly reflects scheduling drift and should not be used to rank the services. Timing the work with several requests in flight gives a more useful comparison, and the performance order depends on the allocator. Therefore, H1 combines round-trip latency, core performance, and overload behaviour into a single claim that the experimental method cannot support.

H2 H2 holds for the seven injected defect classes tested in this study. However, it does not hold under the broader interpretation suggested by its wording. The hypothesis is supported for memory safety and data-race freedom, but not for concurrency correctness in general. It also applies only to *safe* Rust code. The example involving `unsafe` code shows that the same class of defect can still occur when those

guarantees are bypassed.

H3 H3 is supported: any per-call overhead is bounded by 10.91% of the signalling call. This conclusion is based on the upper bound of the paired difference. A non-significant difference alone would not be enough to show that the overhead is small. The additional claim that the main costs are design and training is supported by the structure of the integration, but it was not measured statistically in this study.

6.4 Engineering guidelines for incremental adoption

The following guidelines translate the findings into practical recommendations. Each recommendation should be understood within the limits of the corresponding result.

Choose the first component to migrate based on the simplicity of its API rather than its size. Rewriting a component with a complex interface is more likely to create integration and scheduling risks than rewriting a small component with a simple interface. Keep the C ABI in a separate crate and keep the core free of `unsafe` code. This limits the spread of `unsafe` code through the dependency chain [58].

Do not reduce the budget for concurrency review. Prefer a shared-nothing design where possible, regardless of the language. The strongest concurrency result in this study is primarily a property of the design, not of either compiler.

Treat the learning curve as a real engineering cost and track it explicitly. Measure the component itself as well as the complete round trip. Test workloads beyond the point of saturation, where the campaign can no longer keep up with the offered load. Also audit every operation on the producer's timed path, including histogram correction.

On the tested host, moving this work outside the timed window removed the large high-load latency gaps while leaving a smaller throughput advantage.

6.5 Where this study sits, and what it cannot support

The studies most closely related to this work are the five vendor-hosted theses discussed in Section 2.2. Their conclusions differ substantially from one another.

The study reporting the opposite peak-RAM result can be reconciled with the present findings if the implementations are understood as different concurrency runtimes rather than as direct language comparisons.

Four limitations remain particularly important. First, the same author implemented both versions. This can introduce bias or differences in optimisation effort. A similar problem affected a previous comparison, where the result initially appeared comparable after code review but later showed Rust performing 85–91% worse after additional optimisation work [33]. A parity test suite establishes that the implementations behave similarly, but it does not establish that they have similar performance.

Second, the end-to-end clock measures the system design as a whole, while the actual language-specific component represents only a small part of the measured time.

Third, additional delay samples are written after the timed window. Therefore, they do not contribute to the measured overhead while the timer is running. A throughput advantage remains even when the service can no longer keep up with the offered request rate. The difference in kernel time appears to explain part of this advantage on the tested host, while the queue-swap experiments on another host support the same explanation. However, the earlier run was not repeated in this study.

Fourth, after saturation, the campaign delay mainly represents scheduling drift rather than service performance. It therefore should not be used to rank the services. Timing the actual work with several requests in flight provides a more meaningful comparison, and the resulting order depends on the allocator. This experiment is outside the family of measurements used in the original campaign.

6.6 Ethical and professional reflection

This study uses no personal data, subscriber data, or operator secrets. All traffic is generated specifically for testing. The service is based on live code rather than copied production code, so the resulting artefacts can be shared publicly.

The first professional responsibility is to avoid making claims that the data cannot support. A vendor could interpret the RQ2 result as permission to describe a migrated component as “safe.” However, the seven defect classes that Rust does not prevent show why such a conclusion would be incorrect. A deadlocked control plane can cause an outage, while a silent lost-update bug can lead to incorrect mobility decisions. The results presented here prevent neither problem.

The second responsibility concerns the scope of the evidence. Tail-latency measurements from a single host provide a weaker basis for a production rollout than they may initially appear to provide. This is why the earlier guest-system measurements are retained rather than discarded.

7 Conclusion and Future Work

The thesis set modern C++ against Rust. The object is a UE Context Management service. It keeps per-device state, as a gNB-CU-CP does. It looked at how late the tail is, which defects a compile step can stop, the cost of the safe code model, and the cost of adding Rust a piece at a time. Both sides were built against one frozen spec. They were driven along the same code paths. That is what lets a language effect be told apart from a gap of design or of skill.

The safety mechanisms act at different stages. Safe Rust rejects all seven injected memory-safety and data-race classes before they become programs that can run. The same seven compile in C++, where a sanitiser can report a defect only if a test executes the relevant path. The other seven compile and fail in both languages. On two of them the C++ tools flag what Rust's do not. Rust therefore drops data races and aliasing errors. It does not drop race conditions, lock-order errors, leaks, wrapping arithmetic, or the blast radius of a failed worker. It drops nothing inside an `unsafe` block. LeakSanitizer and UndefinedBehaviorSanitizer flag two of those seven without a rewrite. The two model checkers flag the concurrency cases only after each defect is recast into their parts.

On migration that layer is small. An unchanged Rust core can be driven from C++ across the C-ABI layer where the two languages meet. It has 11 entry points in 231 lines, and its 9 raw-pointer functions contain all of the `unsafe`. The core has 0 `unsafe` sites. The cost on a real signal call is bounded at 10.91% of that call. Prior work identifies API design and the learning curve as adoption costs, but this study does not measure their size. Most of the Rust build's extra cost is building crates, not the compiler's own work. The size gap is a linkage policy. The order rests on which matched policy is chosen. Where C++ leads is the edit and compile loop. It leads only for edits that stay inside one source file.

On speed the data first splits by what is being timed. The end-to-end clock starts before a producer sends a request and stops after the worker replies. It includes the queue, wake-up, and core call. Only 0.250% of that round-trip is the core work the languages perform, so this clock cannot bound a compiler effect. A second measurement removes the queue and cross-thread handoff and times the core call alone. The core is 1.6× faster in Rust in that measurement. At the chosen load of 56 kops/s the paired test finds 1.5 μs in Rust's favour. At p99.99 the slowest Rust variant sits 22 μs below the fastest C++ one. Past saturation Rust still finishes more calls at 784 kops/s. Extra delay samples are written after the timed window. CPU per completed call is 1.03. On one further host, giving Rust

the C++ queue removes that lead, so it comes from how calls pass between threads, not from the core work. Those gaps past saturation are schedule drift. They do not rank the two services. Timing the work with several requests in flight does rank the delay. The rank depends on the allocator, for signal and for the mixed lifecycle. The delay is the wait before queue entry. On RAM the Rust build carries a few MiB more fixed footprint under both allocators. It does so in the same direction and MiB scale on all three hosts timed. The allocator choice costs about as much on both bare-metal hosts, and an order of magnitude more on the guest this study used first.

Taken as a set these support a narrow claim. It is for stateful, concurrent, partitionable control plane services of this kind. At the loads where the service still meets its sent rate, the choice is which classes of defect a team wants to make impossible to write. The other half is whether it will pay the design and training cost of getting them. Past that point the campaign gives no second language rank. Its delay there is schedule drift. The ranking from several requests in flight depends on the allocator, and that run sits outside the tested family. What the thesis adds is the frozen spec, the parity tested pair, and a timing harness that reaches past the saturation region. A split of the round-trip says how much of what it times the language can own. A defect list maps where the guarantee stops.

Future work answers the gaps one at a time. The remaining Rust throughput lead goes away when Rust is given the C++ queue. One job is to repeat that swap on the campaign host. Timing the work with several requests in flight has been run for signal and mixed calls. Repeating it on the campaign host is still open. Another needs no new host. It is to vary how the kernel uses large pages for ordinary memory, on one host with all else held fixed. All three hosts agree that the Rust build carries the larger footprint. They disagree about how that sits against the cost of the allocator. The guest put it an order of magnitude higher, and neither bare-metal host does. The large-page setting is the candidate this study can name but not test, because it was not the same on each host and only the system allocator variant gets it. Beyond that, replaying anonymised production traces would check the call mix. The harness can also take further control plane parts or the data heavy layer-2 and physical layer (PHY) path. A part with truly shared mutable state would work the two sides in a way a shared-nothing design does not. Testing Rust async runtimes against the thread per shard model would split the runtime from the language in the tail result [32]. Timing what a split of the two allocators in one process costs would price what keeping a core free of `unsafe` is worth [62, 63]. A longer case study would track defect counts and review effort while adding Rust a piece at a time. That is what would settle the human cost this study can only name.

Bibliography

[1] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[2] 3GPP. *NR; NR and NG-RAN Overall description; Stage 2*. Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220.

[4] 3GPP. *NR; Radio Resource Control (RRC); Protocol specification*. Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip.

[5] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[6] J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[7] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146.

[8] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation*. BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL.

[9] J. Vander Stoep. *Memory Safe Languages in Android 13*. Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6. URL: https://openlibrary.org/isbn/9781718503106.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154.

[12] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006.

[13] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". In: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438.

[14] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond*. Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en.

[15] AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf.

[16] J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/.

[17] A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/.

[18] R. N. M. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. W. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzinghi, S. W. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553.

[19] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[20] Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf.

[21] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[22] Dell'Oro Group. *RAN Mostly Stable in 3Q 2025*. Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/.

[23] Dell'Oro Group. *RAN Market Stabilized in 2025*. Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index

reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/.

[24] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[25] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036.

[26] A. Meneely, A. Green, T. Jaafari, M. Fluet, and B. Keller. ""Just Use Rust": A Best-Case Historical Study of Open Source Vulnerabilities in C". In: *Proceedings of the 3rd IEEE/ACM International Workshop on Software Vulnerability Management (SVM)*. 2025, pp. 25–32. DOI: 10.1109/SVM66695.2025.00008. URL: https://doi.org/10.1109/SVM66695.2025.00008.

[27] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[28] 3GPP. *NG-RAN; E1 Application Protocol (E1AP)*. Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip.

[29] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[30] V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[31] J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[32] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[33] C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[34] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031.

[35] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[36] F. Suchert and J. Castrillon. "STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks". In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: 10.1007/978-3-031-31180-2_10. URL: https://doi.org/10.1007/978-3-031-31180-2_10.

[37] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. "Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers". In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are

affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[38] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[39] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[40] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[41] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7.

[42] S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.

[43] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[44] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[45] G. L. Steele Jr., D. Lea, and C. H. Flood. "Fast Splittable Pseudorandom Number Generators". In: *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*. 2014, pp. 453–472. DOI: 10.1145/2660193.2660195. URL: https://doi.org/10.1145/2660193.2660195.

[46] L. C. Noll, K.-P. Vo, D. E. Eastlake 3rd, and T. Hansen. *The FNV Non-Cryptographic Hash Algorithm*. Request for Comments (RFC) 9923. Informational. Specifies the FNV-1a hash used inside each shard. Internet Engineering Task Force (IETF), Feb. 2026. DOI: 10.17487/RFC9923. URL: https://www.rfc-editor.org/rfc/rfc9923.

[47] T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard.

[48] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[49] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[50] J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179.

[51] M. Cui, H. Xu, H. Tian, and Y. Zhou. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624.

[52] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. "Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[53] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6. DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494.

[54] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. "Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)". In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. Vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39.

[55] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[56] S. Lu, S. Park, E. Seo, and Y. Zhou. "Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323.

[57] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs". In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642.

[58] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.

[59] A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[60] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[61] W. Crichton and S. Krishnamurthi. “Profiling Programming Language Learning”. In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[62] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. “Keeping Safe Rust Safe with Galeed”. In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[63] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. “PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages”. In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.

Appendix A Host and campaign configuration

Table A.1 itemises the three machines the campaigns ran on: the bare-metal host of Section 5.1 that every performance and resource table of the main sweep is drawn from, the virtualised host it replaced, and the second bare-metal host at a different design that Section 5.1 reads against it. The resident-set decomposition belongs to the second bare-metal host and is reported in Section 5.3.3. The fault catalogue was compiled and run on the development machine, whose toolchain its own caption records. Of the adoption-cost axes only build time is host-sensitive, which is why that one is reported as a paired ratio while the footprint and dependency figures are deterministic. The three small paired experiments predate the move to bare metal and carry no environment record, so each supports a ratio between two variants measured alternately in one session. Section 5.1 points here for the field-by-field record of what each campaign captured. Every column reports what was measured, and the protocol they are read against is published as `bench/TESTBED.md`. Table A.2 gathers the configuration of the service, the workload and the campaign. Each adjacent column pair isolates one change, except where the Kernel and Toolchain rows record otherwise: against the virtualised host the design and container image match but the machine and `g++` point release differ; against the second bare-metal host the shard count, load grid, repeat count and compiler differ too.

Table A.1: The three machines the campaigns ran on, each reported as measured. Every hardware, kernel and toolchain cell is generated from that campaign's own environment capture rather than restated from the protocol of `bench/TESTBED.md`. The operating-system row names the machine and, where a campaign ran in a container, the image's userland beside it, so the kernel row prints one value for both. The last two rows are prose, because what an instrument set is adequate for is a judgement and not a captured field. Rows where all three agree describe the source or the method rather than any machine. The *Short of protocol* marks are deliberate and are argued where they appear.

Aspect	Reported sweep	Virtualised host, same design	Second bare-metal host, different design
Host	AMD EPYC 7443P, 24 cores, 1 socket	Intel(R) Xeon(R) CPU E5-2680 v4 guest, 16 vCPUs	AMD EPYC 7443P, 24 cores, 1 socket
Operating system	Ubuntu 24.04.4 LTS. Image Rocky Linux 8.10 (Green Obsidian)	Rocky Linux 8.10 (Green Obsidian)	Ubuntu 24.04.4 LTS
Kernel	6.17.0-23-generic	7.0.2-1.el8.elrepo.x86_64	6.17.0-23-generic
Hypervisor	none. Clock setting performance, boost off, verified on the machine	kvm. Frequency and turbo host-controlled, unobservable from inside	none. Clock setting performance, verified on the machine
CPU steal	None: no hypervisor to lose a timeslice to	Not characterised. The capture's single one-second sample read 0, and no steal probe was run on this host	0 of 60 intervals on 16 pinned cores
Isolation	`nohz_full` 2-17. `isolcpus` none, omitted on purpose (§5.1)	`isolcpus` none, `nohz_full` none	`isolcpus` none, `nohz_full` none. *Short of protocol*
Large pages	madvise	always, and only the system-allocator variant inherits it	madvise
Toolchain	`g++ 15.2.1`, `rustc 1.97.1`, pinned in a container image	`g++ 15.1.1`, `rustc 1.97.1`	`g++ 13.3.0`, `rustc 1.98`, so a compiler change is tested too
Pinning	Cores 2–17, 7 shards, 7 producers	Cores 2–15, 7 shards, 7 producers	Cores 2–17, 8 shards, 8 producers
SMT	off, verified	Absent from the topology the guest is shown, and decided on the host	off, verified
Repeats	20 over 14 loads, 1120 runs	20 over 14 loads, 1120 runs	15 over 8 loads, 480 runs. *Short of protocol*
Instruments	Counters, pooled HDR sidecars, jitter, allocator accounting	None available in the guest	Counters at two scopes, HDR sidecars, allocator accounting
Adequate for	The pre-committed campaign-clock results. Not overload service latency	Everything except the far tail below saturation	Confirming campaign-clock directions survive a change of design

Table A.2: Configuration of the service under test, the workload driving it, and the measurement campaign. Service parameters are fixed by the frozen specification, with one exception: the shard count is set to match the cores the host offers (Section 5.1). All of them, that one included, are identical in both languages. Campaign figures are generated from the measurement logs together with the tables that report them. The representative load is fixed rather than derived, for the reason Section 5.2 gives. The flat stretch is the lowest of the four, and they differ.

Parameter	Value	Note
Service under test, identical in both languages		
Shards, and worker threads	7	one thread owns one shard. Identical in both languages, which is what the fairness control requires (§3.5)
Queue bound per shard	1 024 requests	bounded and blocking, so overload is observable
Initial map capacity	4 096 entries	reserved so steady state does not rehash
Bearers per context	at most 8	
Signaling payload	64 bytes	
Context ceiling per shard	200 000	
Shard selection	SplitMix64, then modulo	pinned: diffuses sequential identifiers
Intra-shard hash	FNV-1a, 64-bit	pinned on both sides and checked bit-for-bit
Workload		
Workloads	signal, mixed	mixed walks the full RAN lifecycle
Active UEs	5 000	
Offered-load levels	14	up to 784 kops/s
Representative load	56 kops/s	fixed, see caption
Flat throughput	392 kops/s achieved	both languages track through 392 kops/s. Past saturation, extra delay samples are written after the timed window (Section 5.3.2)
Load model	open loop, fixed intended schedule	latency measured from intended send time
Campaign		
Build variants	4	two languages × two allocators
Repeats per sweep point	20	variants interleaved within each repeat
Measured runs	1120	
Latency capture	HDR histogram, [1 ns, 3.6 s], 3 s.f.	same algorithm and configuration on both sides
Summary statistic	median with bootstrap 95% confidence interval	intervals reported in full, not as half-widths
Language comparison	exact paired permutation test	14 tests, Holm-corrected
Attainable p floors	0.0000019 / 0.000027	uncorrected / after correction

What each campaign recorded. Each of the 1120 runs of the factorial design in Table A.2 contributes one newline-delimited JSON record. Nothing is aggregated at capture time. A record carries the configuration (language, allocator, workload, shard and producer counts, target rate), the outcome (operations completed, errors, wall-clock duration, achieved throughput), the latency distribution from the HDR histogram (mean, standard deviation, the quartiles, p50, p90, p99, p99.9, p99.99 and the maximum, in nanoseconds), a shorter block for inter-operation latency variation (mean, standard deviation, p50, p99, p99.9, the maximum and a sample count), the resource counters from `getrusage` (peak resident set size, user and system CPU seconds, CPU utilisation), and the per-operation-kind counts. A `jemalloc` variant adds that allocator's own accounting. A system-allocator variant stores those fields as unavailable rather than as zero, on the same principle as the clock setting and the turbo state. The kind counts are what makes a divergence between the languages show up as a parity failure (Chapter 4) instead of a performance difference. Each run also persists its complete HDR bucket contents to a sidecar, so the repeats can be pooled before a far-tail percentile is read (Section 3.6), and a separate file records `perf` counters at two scopes, the end-to-end path and the operation on its own.

The three small paired experiments have a different shape. For each of 15 interleaved repeats the decomposition records the batch-mean cost in both languages, once for the operation called directly and once through the threaded service. A third dataset of the same shape over 20 repeats records sustained throughput through the bounded queue and completion primitives alone. The FFI comparison of Section 5.3.5 has that structure over 15 repeats. All three predate the environment record, which is why each is read as a ratio between its two variants.

Build time is the one RQ4 axis that is a timing measurement. It is recorded over 15 interleaved repeats at 14 parallel jobs on the development machine, alternating the toolchains so the ratio survives a change of machine even though the seconds do not. The static footprint and the resolved dependency graph are read from the linked binaries and the manifests.

The fault catalogue is categorical. Each of the 14 cases yields four tool logs (AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer on the C++ program, and `rustc` on the safe-Rust counterpart) and one taxonomy row carrying the three sanitiser outcomes, the detection stage, the compiler result, and the expectation registered before the study. The two original cases that safe Rust does not stop yield a second table from the model checkers, and the `unsafe` one a third from the interpreter. The table and figure in Section 5.3.4 are generated from those logs.

Every load sweep writes the environment record of Section 3.7 beside its results, including the fields a given host does not expose. The clock setting and the turbo state are stored as unavailable rather than left out, so the limitation of Section 5.1 is a property of the dataset. The virtualised campaign shares the sweep's design and predates the later instruments: its records carry the percentiles and the resource counters and nothing else. That absence is the measurement-validity argument of Section 5.1, which is why the campaign is kept. The second bare-metal campaign carries the same record shape as the reported sweep, including the allocator accounting that lets Section 5.3.3 decompose the resident set, but runs a different shard count, load grid, repeat count and toolchain. Each campaign has its own generator and macro prefix.

The two timing checks each cover the same fourteen-load, two-language, two-allocator grid. Past saturation, extra delay samples are written after the interval. The processor is AMD EPYC 7443P, the pin is 2–17, and `nohz_full` is 2-17, matching the campaign's AMD EPYC 7443P and `2-17`. The earlier run that wrote none of the extra delay samples was not repeated here and remains an earlier host, so it cannot supply this campaign's percentiles. The overload percentiles of these checks are schedule drift. At the top load Rust is ahead on both allocators, at $p < 0.001$ on the system allocator and $p < 0.001$ under `jemalloc`. CPU per completed operation is 1.03. A throughput lead remains.

The tables that follow are the measurements the main text cites from this appendix.

Table A.3: Inter-operation latency variation at a sustained offered load of 56 kops/s, in microseconds, as a median over 20 interleaved repeats with a bootstrap 95% interval. Measured within one producer and never across producers, since two producers share no schedule and a difference between them is not a variation either one experienced. Reported for completeness rather than as evidence: no research question names this metric, and it is absent from the paired family of Section 3.6.

Variant	p50	p99	p99.9	max
C++ (system alloc)	1.4	22.3	43.6	154.0
	[1.3, 1.5]	[18.6, 33.4]	[39.6, 47.9]	[148.9, 167.0]
C++ (jemalloc)	1.6	22.5	43.7	158.7
	[1.5, 1.6]	[14.3, 31.9]	[42.4, 46.6]	[148.2, 170.0]
Rust (system alloc)	1.5	26.9	40.7	56.9
	[1.3, 1.7]	[20.9, 33.7]	[38.7, 45.1]	[53.4, 107.5]
Rust (jemalloc)	1.6	20.9	44.9	63.4
	[1.6, 1.7]	[15.9, 26.1]	[42.1, 46.6]	[55.7, 108.9]

The allocation profile on the second bare-metal host was taken to name the call sites behind the resident-set difference of Section 5.3.3. Two departures from the campaign explain why it cannot. The first profile ran closed loop at 455 kops/s, past the flat stretch

Table A.4: Where the peak resident set goes on the second bare-metal host, from `jemalloc`'s own accounting in each of the 240 runs of the two `jemalloc` variants. Medians in MiB over the steady-state loads, with the paired language difference in the last column. The two rows below the rule are the argument: the allocator holds *less* in the Rust build, and the process holds more, so the difference is outside the allocator. The final row is the quantity the other rows decompose. The derived row is the median of the per-run difference and not the difference of two medians, because the two figures it subtracts come from one process at one moment, so the column does not sum exactly.

Component	C++	Rust	Difference
Live allocation (`allocated`)	0.78	0.62	−0.16
In active pages (`active`)	1.43	0.79	−0.64
Arena bookkeeping (`metadata`)	4.19	6.26	+2.07
Resident in allocator mappings (`resident`)	11.02	9.10	−1.92
Outside those mappings (derived)	4.04	7.86	+3.82
Peak resident set (`getrusage`)	15.08	16.82	+1.74

bracketed at 448–512 kops/s. Repeating it at a swept load below saturation shrank the discrepancy by most of its size and did not remove it. What remains is that profiling requires an allocator built to sample every allocation and record a backtrace, so the variant under profile is not the variant the campaign measured. Each symbolised profile puts almost all of its flat bytes on a frame that allocates nothing, 98.4% on a deallocation symbol in the C++ variant and 100.0% on the allocator's own backtrace routine in the Rust one. Attributing the footprint would need a profile taken at a swept load, on a build whose peak RSS reproduces the campaign's ordering.